CLAUDE CERTIFIED ARCHITECT - FOUNDATIONS | EXAM CODE CCAR-F

Mock Examination 02

Claude Certified Architect - Foundations
60 scenario-based questions | 120 minutes

Scenarios assessed in this paper: Scenario 1 - Customer Support Resolution Agent; Scenario 2 - Code Generation with Claude Code; Scenario 5 - Claude Code for Continuous Integration; Scenario 6 - Structured Data Extraction

Format: multiple-choice and multiple-response; every question states how many answers to select.

Contents: instructions and blueprint | candidate answer sheet | 60 questions | quick answer key and score tracker | detailed explanations for every option | domain-level review guide | revision checklist.

Status: ORIGINAL PRACTICE MATERIAL. This is not an official Anthropic examination paper and contains no confidential exam content. It was written against the publicly available Claude Certified Architect - Foundations Exam Guide (v1.0, effective July 2026).

Contents

1. Instructions and exam conditions **3**

2. What is verified, what is supplied, and what is original **3**

3. Exam blueprint for this paper **4**

Candidate answer sheet **5**

4. Examination - Questions 1 to 60 **6**

Scenario 1: Customer Support Resolution Agent (Questions 1-15) 6

Scenario 2: Code Generation with Claude Code (Questions 16-31) 10

Scenario 5: Claude Code for Continuous Integration (Questions 32-45) 13

Scenario 6: Structured Data Extraction (Questions 46-60) 17

5. Quick answer key and score tracker **21**

6. Detailed answer key and explanations **22**

7. Domain-level performance review guide **47**

Domain 1 - Agentic Architecture & Orchestration (27% official weight; 16 questions here) 47

Domain 2 - Tool Design & MCP Integration (18% official weight; 11 questions here) 47

Domain 3 - Claude Code Configuration & Workflows (20% official weight; 12 questions here) 47

Domain 4 - Prompt Engineering & Structured Output (20% official weight; 12 questions here) 47

Domain 5 - Context Management & Reliability (15% official weight; 9 questions here) 48

8. Final revision checklist **49**

1. Instructions and exam conditions

Time limit: 120 minutes for 60 questions (an average of 2 minutes per question). Work in one sitting, without notes, documentation, or AI assistance, to get a realistic read on your readiness.

• Every question is anchored to one of the production scenarios in this paper. Read the scenario description at the start of each scenario section; questions are otherwise self-contained.
• Each question states **how many answers to select**. For 'Select ONE', exactly one option is best. For 'Select exactly TWO/THREE', all selected options must be correct; there is no partial credit in this practice format.
• Choose the *best* answer for the stated constraints, not an answer that could merely work. Where a question asks for the FIRST, SAFEST, or MOST RELIABLE step, that qualifier is part of the question.
• Attempt every question. Unanswered questions score zero. There is no 'all of the above' or 'none of the above'.
• Do not look at the answer key until you have finished. The key, explanations, and review guide begin after Question 60.

2. What is verified, what is supplied, and what is original

Category	What it covers
Verified official information	From the official Exam Guide (CCAR-F v1.0): 60 items; multiple-choice and multiple-response items that state how many responses to select; 4 scenarios drawn from a bank of 6; 120-minute limit; scaled passing score of 720 on a 100-1,000 scale; score report with percent-correct by domain; domain weights 27/18/20/20/15%; the 31 task statements and the in-scope/out-of-scope lists.
Scenario descriptions	The six production scenarios are quoted from the official guide; the extra topic lists you were given (e.g., idempotency, structured handoffs, schema design) guided question themes. Scenario text shown above each question group is the official description.
Original practice content	All questions, options, explanations, and review guidance in this file were written for this pack. They are not official questions, are not copied from commercial practice exams or leaked banks, and may differ in difficulty from the real exam.
Supplemental knowledge	A few items use general engineering knowledge to make a scenario realistic (for example CI/CD terminology or least-privilege shell access). They are flagged 'Supplemental' in the explanation and are not confirmed exam objectives.

A raw percentage on this paper is **not** a scaled score. The real exam reports a scaled score (100-1,000) with a cut score of 720 derived from a formal standard-setting study; percent-correct by domain is shown for information only. Doing well on these papers does not guarantee a pass.

3. Exam blueprint for this paper

Domain	Official weight	Questions in this paper
1. Agentic Architecture & Orchestration	27%	16
2. Tool Design & MCP Integration	18%	11
3. Claude Code Configuration & Workflows	20%	12
4. Prompt Engineering & Structured Output	20%	12
5. Context Management & Reliability	15%	9
Total	**100%**	**60**

Question counts are the closest integer allocation of the official weights to 60 questions. Each question has exactly one primary domain; secondary domains appear in the answer key where a question also touches another domain.

Scenario	Questions	Numbers
Scenario 1 - Customer Support Resolution Agent	15	1-15
Scenario 2 - Code Generation with Claude Code	16	16-31
Scenario 5 - Claude Code for Continuous Integration	14	32-45
Scenario 6 - Structured Data Extraction	15	46-60

Design targets (not claims about the real exam): about 20% foundational, 55% intermediate, 25% challenging. In this paper: 12 foundational, 33 intermediate, 15 challenging. Response types: 54 single-answer, 5 select-two, 1 select-three.

Candidate answer sheet

Mark your answers here while taking the paper. Cells that do not exist for a question are greyed out. Where a question says 'Select exactly TWO/THREE', mark exactly that many letters. Do not turn to the answer key until you have finished all 60 questions (or the 120 minutes have elapsed).

Name: _____ Date: _____ Start time: _____ End time: _____

No.	Pick	A	B	C	D	E	F
1	1	(A)	(B)	(C)	(D)		
2	1	(A)	(B)	(C)	(D)		
3	1	(A)	(B)	(C)	(D)		
4	1	(A)	(B)	(C)	(D)		
5	1	(A)	(B)	(C)	(D)		
6	1	(A)	(B)	(C)	(D)		
7	1	(A)	(B)	(C)	(D)		
8	1	(A)	(B)	(C)	(D)		
9	1	(A)	(B)	(C)	(D)		
10	1	(A)	(B)	(C)	(D)		
11	1	(A)	(B)	(C)	(D)		
12	1	(A)	(B)	(C)	(D)		
13	1	(A)	(B)	(C)	(D)		
14	1	(A)	(B)	(C)	(D)		
15	1	(A)	(B)	(C)	(D)		
16	1	(A)	(B)	(C)	(D)		
17	1	(A)	(B)	(C)	(D)		
18	3	(A)	(B)	(C)	(D)	(E)	(F)
19	1	(A)	(B)	(C)	(D)		
20	1	(A)	(B)	(C)	(D)		
21	1	(A)	(B)	(C)	(D)		
22	1	(A)	(B)	(C)	(D)		
23	1	(A)	(B)	(C)	(D)		
24	1	(A)	(B)	(C)	(D)		
25	1	(A)	(B)	(C)	(D)		
26	1	(A)	(B)	(C)	(D)		
27	1	(A)	(B)	(C)	(D)		
28	1	(A)	(B)	(C)	(D)		
29	1	(A)	(B)	(C)	(D)		
30	1	(A)	(B)	(C)	(D)		

No.	Pick	A	B	C	D	E	F
31	1	(A)	(B)	(C)	(D)		
32	1	(A)	(B)	(C)	(D)		
33	1	(A)	(B)	(C)	(D)		
34	2	(A)	(B)	(C)	(D)	(E)	
35	1	(A)	(B)	(C)	(D)		
36	1	(A)	(B)	(C)	(D)		
37	1	(A)	(B)	(C)	(D)		
38	2	(A)	(B)	(C)	(D)	(E)	
39	1	(A)	(B)	(C)	(D)		
40	1	(A)	(B)	(C)	(D)		
41	1	(A)	(B)	(C)	(D)		
42	1	(A)	(B)	(C)	(D)		
43	1	(A)	(B)	(C)	(D)		
44	2	(A)	(B)	(C)	(D)	(E)	
45	1	(A)	(B)	(C)	(D)		
46	2	(A)	(B)	(C)	(D)	(E)	
47	1	(A)	(B)	(C)	(D)		
48	1	(A)	(B)	(C)	(D)		
49	1	(A)	(B)	(C)	(D)		
50	2	(A)	(B)	(C)	(D)	(E)	
51	1	(A)	(B)	(C)	(D)		
52	1	(A)	(B)	(C)	(D)		
53	1	(A)	(B)	(C)	(D)		
54	1	(A)	(B)	(C)	(D)		
55	1	(A)	(B)	(C)	(D)		
56	1	(A)	(B)	(C)	(D)		
57	1	(A)	(B)	(C)	(D)		
58	1	(A)	(B)	(C)	(D)		
59	1	(A)	(B)	(C)	(D)		
60	1	(A)	(B)	(C)	(D)		

4. Examination - Questions 1 to 60

Time allowed: 120 minutes. Start the clock now.

Scenario 1: Customer Support Resolution Agent (Questions 1-15)

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Question 1

Select ONE answer.

Policy says the support agent may issue refunds up to $500 on its own; anything larger needs a human decision. Despite a clear system prompt rule, about 3% of refund calls above $500 are still executed by the agent. The team wants the policy to hold every time while keeping customers moving. What should they implement?

A. A stronger system prompt that repeats the $500 limit in capital letters at the start and end of the instructions and adds two examples of correct behavior.

B. A pre-call hook on process_refund that blocks amounts above $500 and redirects the case to the escalate_to_human workflow with the case details.

C. A nightly report listing refunds above $500 so a manager can reverse any that should not have been issued autonomously.

D. A requirement that the agent self-report its confidence before each refund and skip any refund where confidence is below 90 percent.

Question 2

Select ONE answer.

In testing, Claude sometimes responds with two tool_use blocks in one turn: get_customer and lookup_order, which are independent. The developer's loop executes only the first block and sends back one tool_result. The next API call fails or the agent behaves strangely. What is the correct handling?

A. Treat a response with more than one tool_use block as an error and ask the model to retry with a single tool call.

B. Combine both results into a single tool_result block tied to the first tool_use ID and ignore the second ID.

C. Execute every tool_use block in the response and return a matching tool_result for each tool_use ID in the next message, then continue the loop.

D. Execute only the first tool_use block and discard the second, since the model will request it again on the next turn if the result is still needed.

Question 3

Select ONE answer.

Order lookups sometimes return payment details including full card numbers, and the model then repeats them in its replies. Compliance wants the model never to see full card numbers, while it still needs the last four digits to confirm which card was charged. Which design best meets this?

A. A PostToolUse hook that masks card numbers down to the last four digits in tool results before the model processes them.

B. A system prompt instruction telling the agent never to write a full card number in any reply and to mask digits when it does mention a card.

C. Remove lookup_order from the agent's tools and have staff paste payment details into the chat when needed.

D. A post-processing filter on the final customer message that removes any 16-digit sequence before the reply is sent.

Question 4

Select ONE answer.

A support organization runs a weekly quality audit of closed tickets. Every ticket must be checked for the same four aspects: policy compliance, tone, correct tool use, and accuracy of the resolution. Reviewers want consistent, repeatable results. Which decomposition fits best?

A. A dynamic plan where the model decides at runtime which aspects to check for each ticket and in what order.

B. Four independent agents with their own tools that never share findings and a coordinator that picks the one with the longest report.

C. A single request asking for a complete audit of all four aspects in one response, with a list of criteria at the end.

D. A fixed sequential prompt chain with one focused step per aspect, each step receiving the ticket and the previous findings as needed.

Question 5

Select ONE answer.

The agent calls process_refund and the gateway responds slowly. The agent's tool wrapper times out, retries the call, and the customer is refunded twice. Logs show the first call actually succeeded. What is the most reliable fix?

A. Remove the retry logic so that any timeout is reported to the customer as a failure and left for a human to resolve.

B. Have the tool layer attach an idempotency key derived from the order and request, so repeated calls for the same refund return the original result instead of paying again.

C. Add a system prompt rule telling the agent to call lookup_order before every retry to check whether the refund was already issued, and to skip the retry if it finds a matching refund.

D. Lower the timeout so the wrapper gives up faster and reports failures sooner.

Question 6

Select ONE answer.

A customer asks about three separate orders in one message: delivery status for order A, a return on order B, and a billing question on order C. The lookup for order C times out while A and B succeed. Which behavior is best?

A. Reply only about the order that succeeded first and ignore the other two to keep the answer short.

B. Hold the entire reply until the lookup for order C succeeds, retrying silently in the background, so that the customer receives one complete and consistent answer at the end of the conversation.

C. Handle the concerns as separate items, answer A and B with their results, and clearly tell the customer that C could not be checked yet, offering a retry or escalation.

D. Report that all three lookups failed so the answer is consistent, and ask the customer to try again later.

Question 7

Select ONE answer.

A team is deciding which of its support agent requirements to implement as hooks versus prompt instructions. Which requirement is best enforced with a hook?

A. Replies should sound warm and empathetic while staying concise.

B. The agent should summarize what it did at the end of each conversation.

C. Refunds over $500 must be routed to a human instead of being issued by the agent.

D. When an order number is mentioned, the agent should generally prefer lookup_order over get_customer.

Question 8

Select ONE answer.

A customer asks, 'Where are my recent orders?' The order service is down, and the MCP tool returns an empty list marked as success. The agent tells the customer they have no orders. What is the best tool design?

A. Keep returning an empty list in both cases, but add an optional free-text note field that the service fills in on some errors to explain to the agent what went wrong.

B. Return an error result with category transient, isRetryable true, and a description, reserving an empty successful result for a lookup that ran and found nothing.

C. Return the HTTP status code number as the tool output and let the model interpret it.

D. Have the agent call the tool three times and treat three empty results as proof the customer has no orders.

Question 9

Select ONE answer.

Your team wants the agent to read and update tickets in a standard ticketing product, and also to issue refunds under your company-specific approval rules. Developers propose building both integrations as custom MCP servers. What is the best approach?

A. Skip MCP and have the agent call each system through Bash with curl commands that developers write by hand.

B. Build custom servers for both integrations so that every tool description is written in the team's own style and wording, no outside code is trusted, and the team controls every update.

C. Use community servers for both and encode the refund approval rules as a long section in the system prompt.

D. Use an existing community MCP server for the standard ticketing integration and build a custom server only for the refund logic, which is specific to your workflows.

Question 10

Select ONE answer.

Support leads want the agent to label each billing dispute with a priority from 1 to 3. Today the same dispute gets different priorities on different runs, and the label definitions in the prompt are 'high priority means important and urgent'. Which improvement is most effective?

A. Ask for a numeric confidence score alongside each priority and use it to resolve disagreements between runs.

B. Run each dispute through the classifier five times and use the most common label.

C. Ask the agent to think carefully about urgency, to compare each new dispute with how it labeled earlier disputes, and to keep its priority labels consistent from one dispute to the next.

D. Define each priority with explicit criteria and a concrete example case (for example amount thresholds, account standing, and time sensitivity) so classification is consistent.

Question 11

Select ONE answer.

A customer asks the agent to match a lower price they found at a competitor's store for an item bought last week. The company policy only describes adjustments for its own price drops within 14 days. What should the agent do?

A. Approve the match under the own-site price adjustment rule since it is the closest rule available.

B. Escalate to a human, because the policy is silent on competitor price matching, and include the facts of the request in the handoff.

C. Offer a goodwill credit for a modest amount to settle the conversation and avoid involving others.

D. Decline the request because it is not covered, and close the case so the customer can look for other options.

Question 12

Select ONE answer.

A customer says, 'I'm Maria Garcia, I need my last order refunded.' get_customer returns three accounts for 'Maria Garcia' in different cities. What should the agent do next?

A. Proceed with the first account returned and mention to the customer which one was used.

B. Ask the customer for an additional identifier, such as the email address or postal code on the account, before proceeding.

C. Select the account with the most recent order, since that is the most likely one the customer means, and proceed with the refund on it right away.

D. Escalate immediately to a human because a common name makes the case too ambiguous to handle.

Question 13

Select ONE answer.

A customer writes angrily that a package is three days late and demands that something be done, but does not ask for a person. The agent can see the order, confirm the delay, and offer a replacement or refund under policy. What is the best response?

A. Escalate immediately because strong negative sentiment indicates the case is too sensitive for an automated agent to handle without a person involved.

B. Ask the customer to rate their frustration from 1 to 10 and escalate if the number is above 7.

C. Ignore the tone and respond only with the order status so the exchange stays factual.

D. Acknowledge the frustration, offer the replacement or refund the policy allows, and escalate only if the customer then insists on a human.

Question 14

Select ONE answer.

A coordinator gathers results from three parallel investigations (shipping, billing, and account history) into one prompt for the final response. The final reply often omits a key billing finding that appears in the middle of the combined text. What is the best way to organize the aggregated input?

A. Increase the amount of detail in every section so the important finding is repeated several times throughout.

B. Ask the final agent to re-read the combined text ten times before writing the response.

C. Append the three results in random order each run so that no investigation is systematically placed in the middle.

D. Put a short summary of the key findings at the beginning and organize the detailed results under explicit section headers.

Question 15

Select ONE answer.

The billing and shipping investigators each return a long narrative with reasoning steps. The coordinator's context fills quickly and its final synthesis loses detail. The coordinator needs only amounts, dates, statuses, and a relevance rating. What change is best?

A. Increase the coordinator's context by moving to a larger-window model, and keep the narratives unchanged so that no reasoning detail is thrown away.

B. Have the coordinator truncate each narrative to its first 500 characters before reading it.

C. Change the investigators to return structured data, such as key facts, citations, and relevance scores, instead of long reasoning narratives.

D. Add a summarizer subagent that rewrites every narrative into a paragraph before the coordinator sees it.

Scenario 2: Code Generation with Claude Code (Questions 16-31)

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Question 16

Select ONE answer.

You spent Monday debugging a payment timeout in Claude Code in a session you named 'payment-timeout'. On Tuesday you want to continue that exact investigation with its accumulated context. What is the right way to do it?

A. Resume the named session with --resume payment-timeout and continue from where you stopped.

B. Run /init so Claude regenerates its understanding of the repository from scratch.

C. Create a new CLAUDE.md note that says 'continue the payment-timeout investigation' and start a new session.

D. Start a fresh session and paste the terminal scrollback from Monday so Claude can reconstruct the context.

Question 17

Select ONE answer.

A service has an intermittent failure that happens about once a day with no obvious pattern. A teammate suggests giving Claude Code a fixed checklist: 1) read logs, 2) check database, 3) check network, 4) report. Previous investigations stalled when the real cause was a dependency nobody listed. Which approach is better?

A. Split the checklist into four parallel agents and merge their reports without further investigation.

B. Use an adaptive investigation: map the system, form hypotheses from the evidence, and generate the next subtasks based on what each step reveals.

C. Keep the fixed checklist but extend it to 40 items covering every plausible dependency, and have Claude work through the items strictly in order each time.

D. Ask Claude to propose a complete root cause immediately from the logs and then verify it at the end.

Question 18

Select exactly THREE answers (choose 3).

A team lead is reviewing practices for long-running Claude Code investigations. Select the THREE statements that are correct.

A. Resuming is always more reliable than starting fresh because the full history is preserved.

B. After code changes, a resumed session should be told which files changed so it can re-analyze them rather than re-explore everything.

C. When most earlier tool results are stale, starting a new session with a structured summary is more reliable than resuming.

D. fork_session can create independent branches from a shared analysis baseline to compare two refactoring approaches.

E. fork_session creates an isolated copy of the repository so the branches can never affect each other's files.

F. A resumed session automatically re-reads any file that has changed on disk since the session was last used.

Question 19

Select ONE answer.

A developer asks Claude Code, 'Which Jest test files import from billing/utils?' The repository has about 6,000 files. Which approach is the most direct?

A. Glob for **/billing/utils* to find all files with that name and report them.

B. Use Bash with find and cat to print every file in the repo and search the output manually.

C. Glob for **/*.test.* and then Read every matching test file to look for the import.

D. Grep for the import path pattern across the codebase and keep the matches in test files.

Question 20

Select ONE answer.

You are new to a 3,000-file monolith and need to understand how checkout applies discounts. Which exploration strategy is best for Claude Code?

A. Grep for the discount entry point, Read that file, then follow its imports to trace the flow step by step.

B. Read every file under the checkout directory first so that Claude holds the whole module in context before answering.

C. Ask Claude to explain how checkouts typically apply discounts in similar applications and rely on that description.

D. Glob for all files with 'discount' in the name and treat that list as the complete implementation.

Question 21

Select ONE answer.

The shared .mcp.json defines a GitHub server whose token comes from ${GITHUB_TOKEN}. Everyone else on the team sees the GitHub tools in Claude Code, but one teammate sees none and the server shows as failed. They cloned the same repository. What should they check first?

A. Whether the teammate should reinstall Claude Code because the project-level configuration is only read by fresh installs.

B. Whether the GitHub server definition should be moved into CLAUDE.md so Claude Code reads it as an instruction.

C. Whether the token should be pasted directly into .mcp.json for that teammate and committed so the config matches everyone's.

D. Whether GITHUB_TOKEN is set in the environment of the shell that launches Claude Code, since the config expands that variable at startup.

Question 22

Select ONE answer.

Your payments/ directory has stricter coding rules (decimal arithmetic only, mandatory audit logging) that matter nowhere else. You want Claude Code to apply them when working there, shared with the whole team. Where should they go?

A. In each developer's ~/.claude/CLAUDE.md so the rules are always loaded whichever project they open.

B. In the root CLAUDE.md so that every instruction exists in a single file loaded everywhere.

C. In a custom command that developers run before editing anything in payments/.

D. In a CLAUDE.md inside the payments/ directory, committed to the repository.

Question 23

Select ONE answer.

Three tests fail after a change. All three trace back to one shared cache that serializes dates differently from how the parser reads them, so fixing one often changes the others. How should you give feedback to Claude Code?

A. Report only the first failure and wait to see whether the others disappear, ignoring them until then.

B. Fix them one at a time in three separate sessions, so each fix starts with a clean context and cannot be influenced by the other two.

C. Share all three failures together in a single detailed message so Claude can address the interacting cause at once.

D. Ask Claude to disable the three failing tests until the cache work is complete.

Question 24

Select ONE answer.

A rule file .claude/rules/api.md has frontmatter paths: ["src/api/*"]. Claude applies it when editing src/api/index.ts but ignores it when editing src/api/v2/users/handler.ts. How should the paths pattern be changed?

A. Use a recursive glob such as src/api/**/* so files in nested directories match as well.

B. Remove the paths field entirely so the rule loads on every request regardless of the file being edited.

C. Move the conventions into a skill and ask developers to invoke it for deeper directories.

D. Add a separate rules file for each nested directory and list the same conventions in every one of them.

Question 25

Select ONE answer.

Two developers use the same repository and project CLAUDE.md, yet Claude Code follows an 'avoid default exports' convention for one and not for the other. You suspect a difference in what is loaded. What is the best way to diagnose it?

A. Ask both developers to reinstall Claude Code and clone the repository again from scratch, before anything else, to rule out a corrupted installation or checkout.

B. Compare both developers' Claude Code versions and downgrade the newer one.

C. Add the convention three more times to the project CLAUDE.md so it is certain to be noticed.

D. Run /memory in both sessions to see which memory files are loaded, then compare, for example to find a rule that exists only in one developer's user-level file.

Question 26

Select ONE answer.

A teammate asks why Terraform conventions should go in .claude/rules/terraform.md with paths: ["terraform/**/*"] instead of the root CLAUDE.md. What is the main advantage?

A. Rules files can be invoked by typing a slash command, which makes them easier to find.

B. Rules files take priority over CLAUDE.md instructions whenever the two disagree about the same topic, which makes them safer for strict conventions.

C. The rules load only when working on matching files, which keeps irrelevant instructions out of context and reduces token usage.

D. Rules files run faster than CLAUDE.md because they are compiled into a binary format.

Question 27

Select ONE answer.

A /fix-ticket skill requires a ticket ID, but developers often run it with no argument, and Claude then guesses or asks vaguely. What is the simplest improvement in the skill file?

A. Add a sentence to CLAUDE.md saying every skill must be invoked with all required parameters.

B. Add an argument-hint to the skill's frontmatter, such as [ticket-id], so developers are shown the required parameter when they invoke the skill.

C. Add context: fork to the skill's frontmatter so that it runs in a sub-agent, which can then ask the developer for the missing ticket ID.

D. Move the skill to ~/.claude/skills/ so developers each see the prompt privately.

Question 28

Select ONE answer.

Three tasks are queued: (X) fix a null pointer crash with a clear stack trace pointing to one method, (Y) add social login where two approaches (a hosted identity provider or a self-managed OAuth flow) require different infrastructure, (Z) rename a configuration key used in a single file. Which assignment of mode is best?

A. X direct execution, Y direct execution, Z direct execution, adjusting if complications appear.

B. X plan mode, Y plan mode, Z direct execution.

C. X direct execution, Y plan mode, Z direct execution.

D. X plan mode, Y plan mode, Z plan mode, so every change follows the same process.

Question 29

Select ONE answer.

You need a rate limiter with specific behavior for bursts, a requirement for constant-time checks, and several edge cases (clock skew, zero quota). Past attempts at describing it in prose produced code that missed edge cases. What workflow is likely to work best?

A. Describe the requirement in more detail and ask Claude to say 'done' only when it is confident it is correct.

B. Generate three independent implementations and keep the one with the fewest lines.

C. Write the tests first covering expected behavior, edge cases, and performance, then have Claude implement and iterate by sharing test failures.

D. Ask Claude to write the implementation first and then write tests that match whatever the implementation does, so that the tests are guaranteed to pass.

Question 30

Select ONE answer.

Midway through a large refactor, the main Claude Code session is at roughly 70% of its context after repeated exploration. You need to find all test files for the billing module and trace the refund flow's dependencies, then continue coordinating the refactor. What approach protects the main session best?

A. Run both investigations directly in the main session so the details stay in view, and use /compact only when the context actually overflows its limit.

B. Delegate each question to a subagent that explores in isolation and returns a concise summary, keeping the main session for coordination.

C. Ask Claude to skip tracing and refactor based on its general knowledge of billing systems.

D. Clear the conversation and re-explain the refactor goals from memory, then run the investigations.

Question 31

Select ONE answer.

A long exploration session has filled with verbose search output, and responses are slowing and getting less precise. The important findings are already written into a notes file. What is the best next step?

A. Keep working until the session errors, then start over without the notes file.

B. Open a second terminal and run the same session in both windows so the load is shared.

C. Delete the notes file so Claude must rediscover the findings and therefore pays closer attention.

D. Use /compact to reduce context usage, and keep referring to the notes file for the specifics.

Scenario 5: Claude Code for Continuous Integration (Questions 32-45)

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question 32

Select ONE answer.

Your CI platform needs two Claude-powered jobs: (1) a nightly report that always checks every merged PR for the same four aspects (security, tests, docs, performance notes), and (2) an on-demand investigation of an unexplained latency regression whose cause is unknown. Which decomposition pairing is best?

A. A fixed sequential prompt chain for the nightly report, and dynamic adaptive decomposition for the latency investigation.

B. Dynamic adaptive decomposition for the nightly report, and a fixed prompt chain for the latency investigation.

C. Dynamic adaptive decomposition for both jobs so each run is tailored to the repository.

D. A fixed prompt chain for both jobs so behavior is predictable and easy to maintain across the platform.

Question 33

Select ONE answer.

A pull request changes 12 files. You run one focused review per file in parallel, then a final integration pass. The integration pass currently receives the full text of all 12 files again and produces shallow output. What should it receive instead?

A. The full text of all 12 files again, plus a longer prompt asking it to examine every file as carefully as the local passes did, in a single pass.

B. Only the list of file names, so that the integration pass is not biased by the local findings.

C. The structured per-file findings plus a compact description of changed interfaces and data flows, with the full code of only the files involved in each cross-file concern.

D. Nothing at all, running the integration pass on the PR title to keep it independent of earlier passes.

Question 34

Select exactly TWO answers (choose 2).

Select the TWO workflow-to-decomposition pairings that are well matched.

A. Generating the same weekly summary format for every team, using a fully dynamic plan that the model invents on each run.

B. Adding tests to a poorly understood legacy module, by mapping the structure, then prioritizing and adapting the plan as dependencies appear.

C. Reviewing a 14-file PR, using one pass over all files with a prompt asking for equal depth on each.

D. Reviewing every PR for the same fixed list of concerns, using a prompt chain with one step per concern.

E. Diagnosing an unfamiliar production incident, using a fixed five-step script written before the incident.

Question 35

Select ONE answer.

In CI, Claude Code uses an MCP tool to fetch repository metadata. The CI token lacks a required scope, so the tool returns 'Request failed'. The job retries it five times and then times out the stage. What tool behavior would prevent this waste?

A. Return the same 'Request failed' message but include a timestamp so engineers can find the matching server log.

B. Return an empty metadata object so the review proceeds without metadata and appears to succeed.

C. Add a system prompt instruction that the agent should retry any failing tool call at most five times.

D. Return an MCP error with category permission, isRetryable false, and a description naming the missing scope, so the agent stops and reports the cause.

Question 36

Select ONE answer.

During review, the code-search MCP tool is rate limited and returns an empty result. The reviewer concludes 'no other callers of this function exist' and approves its deletion. The function is used in four places. What is the best fix on the tool side?

A. Return an empty result in both cases, but add a warning sentence to the system prompt explaining that the tool can be rate limited and may return nothing.

B. Lower the reviewer's authority by labeling every deletion suggestion as advisory, regardless of the evidence.

C. Make the reviewer run every search twice and accept the deletion only if both runs are empty.

D. Return an error indicating the search did not complete, with the failure type and whether retrying is advised, instead of an empty success.

Question 37

Select ONE answer.

Claude Code in CI generates unit tests, but they use ad hoc mocks instead of the team's shared fixtures and often test trivial getters. Where should you document the team's testing standards, valuable-test criteria, and available fixtures so CI runs follow them?

A. In a comment at the top of each test file, since Claude only reads the file it is currently editing.

B. In the project's CLAUDE.md, which provides project context to Claude Code including when it is invoked from CI.

C. In a developer's ~/.claude/CLAUDE.md, because CI picks up the personal file of whoever triggered the build.

D. In the pipeline's job log, so that the output of earlier runs is visible to future runs automatically.

Question 38

Select exactly TWO answers (choose 2).

Your CI re-runs Claude Code review after each new commit on a PR. Developers complain that old comments are repeated, and the test generator keeps suggesting tests that already exist. Select the TWO changes that best fix this.

A. Run the review only on the final commit and disable it on every earlier commit.

B. Delete all earlier review comments before every re-run so the PR starts clean each time.

C. Raise the sampling temperature so repeated runs naturally produce different wording and different suggestions.

D. Provide the existing test files in context so test generation avoids suggesting scenarios already covered.

E. Include the prior review findings in the context and instruct Claude to report only new or still-unaddressed issues.

Question 39

Select ONE answer.

A CI step posts inline PR comments by parsing Claude Code's markdown output with regular expressions. The parser breaks whenever Claude changes a heading or bullet style. What is the more reliable approach?

A. Run Claude Code with --output-format json and a --json-schema describing the findings so output is machine-parseable.

B. Ask Claude to output the findings twice in different formats and use whichever one parses successfully.

C. Switch the pipeline to interactive mode so a person can copy comments into the PR by hand.

D. Add stricter instructions in the prompt about exactly how to format headings and bullets, and keep the regular expressions.

Question 40

Select ONE answer.

The same Claude Code session writes a feature and then, in the same session, is asked to review its own changes. It consistently approves them even when subtle defects exist. What is the better CI design?

A. Run the review in a separate, independent Claude Code invocation that has not seen the generation reasoning, receiving only the diff and project context.

B. Skip review for changes Claude generates because they were already written with care.

C. Keep the same session and tell Claude to act as a strict external reviewer that has never seen the code before, using all the project's review criteria.

D. Ask the same session to review three times and take the most critical of the three reviews.

Question 41

Select ONE answer.

A CI step requires Claude's review output to match a JSON schema. Occasionally the output omits the required 'severity' field for a finding. The diff and finding data are otherwise fine. What is the best recovery?

A. Retry the original request unchanged up to ten times and accept the first valid response.

B. Fill the missing field with a default value of 'low' in code so that the pipeline always receives a valid object and the job continues without interruption.

C. Fail the CI job on the first validation error, since invalid output means the model is unreliable.

D. Retry with a follow-up that includes the original diff, the failed output, and the specific validation error, asking Claude to correct it.

Question 42

Select ONE answer.

The review tool uses tool_use with a JSON schema, so every response is valid JSON. Still, some findings point to line numbers outside the PR's diff, producing comments on unrelated lines. Which control catches this?

A. A programmatic check that each finding's file and line fall within the diff hunks, with invalid findings rejected or sent back for correction.

B. A stricter JSON schema that makes the line number field required, an integer, and greater than zero, so that obviously invalid numbers cannot be returned.

C. Switching tool_choice from 'any' to 'auto' so the model can decide when to output findings.

D. A prompt addition reminding the model that line numbers must be correct and carefully checked.

Question 43

Select ONE answer.

A weekly security audit sends 2,000 source files to Claude, takes place over the weekend, and results are read on Monday. During the audit, the model needs to call a Grep tool mid-analysis to follow references between files. Your team wants the 50% batch discount. What is the correct evaluation?

A. Use the Message Batches API for the weekly audit, but gather the needed cross-file context before submission, because a batch request cannot call tools mid-request.

B. Avoid batches because the audit needs tools, and run it synchronously to avoid any restrictions.

C. Use the Message Batches API and let each request call Grep during processing as needed, since tool use works the same way in batches and returns results mid-request.

D. Use batches for pre-merge checks as well, since 50% savings apply to any workload that calls the API.

Question 44

Select exactly TWO answers (choose 2).

You are designing the JSON schema for review findings. Select the TWO practices that best prevent fabricated or unusable values.

A. Make suggested_fix nullable so that the model can return null when there is no concrete fix, instead of inventing one.

B. Use a free-form string for severity so the model can describe it in its own words.

C. Make every field required so the model must always provide complete information for each finding.

D. Rely on tool_choice 'auto' to guarantee that findings always come back through the schema.

E. Use an enum for category with an 'other' option and a free-text detail field for unlisted types.

Question 45

Select ONE answer.

Documents arrive continuously and your SLA is that every result must be available within 30 hours of arrival. Batches can take up to 24 hours to process. You submit all accumulated documents on a fixed schedule. Which submission interval guarantees the SLA?

A. Every 12 hours, which keeps costs low by creating fewer, larger batches.

B. Every 24 hours, matching the maximum processing window so the schedule is easy to remember.

C. Every 4 hours, because a document waits at most 4 hours for the next submission plus up to 24 hours of processing.

D. Every 8 hours, because a document waits at most 8 hours and batches usually finish well before 24 hours.

Scenario 6: Structured Data Extraction (Questions 46-60)

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Question 46

Select exactly TWO answers (choose 2).

An extraction agent calls an OCR tool that returns dates in mixed formats, then calls write_to_ledger to post records downstream. Select the TWO hook-based controls that fit this design.

A. A system prompt paragraph asking the agent to remember to validate each record before writing it.

B. A hook that increases max_tokens whenever the OCR output is long so the model can reason more.

C. A hook that rewrites the model's extracted vendor names to what it believes they should be, based on fuzzy matching.

D. A pre-call hook on write_to_ledger that blocks the call unless the record has passed schema and business-rule validation.

E. A PostToolUse hook that normalizes OCR dates into a single ISO format before the model processes the result.

Question 47

Select ONE answer.

Finance rules say that extracted invoices with a total above $10,000 must never post automatically to the ledger and instead go to a manual approval queue. Which implementation guarantees this?

A. A line in the system prompt stating clearly that invoices with a total over $10,000 must never be posted and must go to the approval queue instead.

B. A tool-call interception hook that checks the total on write_to_ledger and, above the threshold, redirects the record to the approval queue.

C. A few-shot example showing a large invoice being sent to the queue, added to the extraction prompt.

D. A daily report of ledger entries above $10,000 for finance to review the next morning.

Question 48

Select ONE answer.

A 120-page contract is processed in one request, and the extraction of obligations is thorough for early sections but patchy for later ones. Some obligations in one section contradict definitions in another. How should the workflow be restructured?

A. Extract obligations section by section in focused passes, then run a separate reconciliation pass that checks cross-section definitions and conflicts.

B. Use only the first 40 pages, where extraction quality is highest, and mark the remaining pages as out of scope.

C. Run the same single request three times and merge the obligations that appear in at least two runs.

D. Keep the single request but ask the model to give every section equal attention and to double-check later sections.

Question 49

Select ONE answer.

Your extraction service defines three tools: extract_invoice, extract_receipt, and extract_contract. Incoming documents have unknown types. Sometimes Claude replies with text like 'This looks like an invoice' rather than calling any tool, and the pipeline receives no structured data. What configuration fixes this?

A. Set tool_choice to 'any' so Claude must call one of the extraction tools, while still choosing which one fits the document.

B. Force the extract_invoice tool for every document, regardless of type, so that the pipeline always receives structured data in one predictable schema.

C. Remove two of the three tools and use one generic tool that accepts any document type.

D. Keep tool_choice 'auto' and add a sentence to the prompt asking Claude never to answer with text.

Question 50

Select exactly TWO answers (choose 2).

Two extraction tools, extract_data and parse_document, have nearly identical descriptions, and the agent calls them interchangeably, producing inconsistent output shapes. Select the TWO changes that best fix the ambiguity.

A. Split the overlapping responsibilities into purpose-specific tools (for example extract_invoice_fields and parse_layout_to_text), each with a defined input and output contract.

B. Rename the tools to purpose-specific names and rewrite their descriptions with inputs, outputs, and when to use each.

C. Add a third tool called analyze_everything that wraps both so the model has a safe default.

D. Reduce both descriptions to a single word so the model relies on tool names alone.

E. Instruct the model in the system prompt to alternate between the two tools to even out usage.

Question 51

Select ONE answer.

The extraction pipeline must always run extract_metadata first (document type, language, date) and then call enrichment tools that depend on that metadata. Sometimes the model skips straight to enrichment. How do you guarantee the order?

A. Force extract_metadata on every request in the conversation so it always runs before anything else.

B. Set tool_choice to 'any' on the first request so the model must call a tool, and trust it to pick the right one.

C. Force extract_metadata with tool_choice on the first request, then continue in follow-up turns once its result is available.

D. Describe the required order in the system prompt and list extract_metadata first in the tools array.

Question 52

Select ONE answer.

Credit notes are being sent to extract_receipt by the agent and stored with positive amounts, although they should go to extract_invoice and keep their negative sign. Both tools have one-line descriptions. What is the best fix?

A. Merge the two tools into one extract_financial_document tool with a type enum and a one-line description.

B. Tell the agent in the prompt to prefer extract_invoice whenever the document contains any amount.

C. Expand the descriptions with input formats, example documents, edge cases such as credit notes, and boundaries stating when to use each tool.

D. Add a regular expression that looks for the phrase 'credit note' and switches the tool call in application code after the model chooses.

Question 53

Select ONE answer.

A schema requires po_number. For invoices that only reference a purchase order by an attached document not provided to the model, validation keeps failing and the retry loop produces invented PO numbers that pass the format check. What is the best change?

A. Add a regular expression check that rejects PO numbers that don't match the format, and retry on mismatch.

B. Make po_number nullable and route invoices without it to review, instead of retrying, since the information is absent from the provided source.

C. Add more retries with the validation error attached each time, for every invoice, until the model finally supplies a PO number that passes the format check.

D. Ask the model to use the invoice number as the PO number whenever it cannot find one.

Question 54

Select ONE answer.

You submitted a batch of 100 documents. Results show 93 succeeded and 7 failed because the documents exceeded the context limit. What is the best next step?

A. Identify the failures by custom_id, split those 7 documents into chunks, and resubmit only them.

B. Discard the 7 failed documents because batch processing is best effort.

C. Resubmit the entire batch of 100 unchanged so the results are consistent.

D. Move all 100 documents to the synchronous API without changes to handle failures faster.

Question 55

Select ONE answer.

Extracted invoices sometimes have line items that do not add up to the stated total, yet the JSON validates against the schema. Which design best surfaces these cases?

A. Ask the model to double-check that its numbers add up before it responds.

B. Extract the stated total as printed and also a calculated total from the line items, and flag any discrepancy for review.

C. Replace the stated total with the sum of line items in all cases, as the sum is always more reliable.

D. Make the total field required and numeric, and add a minimum value of zero, so that the model cannot omit it or return something invalid.

Question 56

Select ONE answer.

A teammate claims, 'Because we use tool_use with a strict JSON schema, extraction output can no longer be wrong.' Which statement best corrects this?

A. Strict schemas guarantee both valid structure and correct values as long as every field is marked required.

B. Strict schemas via tool use eliminate JSON syntax errors, but values can still be semantically wrong, for example in the wrong field or inconsistent with each other.

C. Strict schemas only apply to the first request in a conversation and are ignored in later turns.

D. Strict schemas help only when the prompt contains few-shot examples; otherwise the output may not be valid JSON.

Question 57

Select ONE answer.

A purchase order's header says delivery date 2026-03-04 but the footer says 2026-03-14. The extraction tool picks the header date and the record flows downstream without comment. Later it causes a late shipment. What is the best design?

A. Always prefer the date that appears first in the document, because headers are usually authoritative and footers are often boilerplate text copied from templates.

B. Include a conflict_detected flag, record both values with their locations, and route the record to review instead of silently choosing one.

C. Make the field required and add a validator that rejects any date not in ISO format.

D. Retry the extraction until the model returns a consistent date for the field.

Question 58

Select ONE answer.

The document_type enum accepts invoice, receipt, contract, and purchase_order. Credit memos and other unforeseen documents get forced into 'invoice' and corrupt downstream reports. What schema change is best?

A. Add an 'other' value with a document_type_detail string, and an 'unclear' value for ambiguous cases.

B. Reject any document whose type is not in the enum and drop it from the batch.

C. Change document_type to free text so the model can write whatever it wants.

D. Add a new enum value for every document type the team can imagine, and update it each quarter.

Question 59

Select ONE answer.

An extraction system reports 97% field-level accuracy overall, and management proposes removing human review for all high-confidence extractions. A closer look shows most validation samples are typed invoices. Which step should come before automating?

A. Raise the confidence threshold slightly and proceed with full automation, since 97% accuracy is above the usual target for automated extraction.

B. Ask the model to rate its own confidence for the whole batch and automate if the average is high.

C. Measure accuracy segmented by document type and field, including low-volume types such as handwritten receipts, before reducing human review.

D. Remove review for typed invoices and keep it for everything else, based on the overall accuracy.

Question 60

Select ONE answer.

Your pipeline auto-accepts extractions whose field confidence exceeds a threshold calibrated on a labeled validation set. Six months later, an upstream vendor changes its invoice layout and errors in auto-accepted records rise without anyone noticing. What practice would have detected this?

A. Stratified random sampling of high-confidence extractions for ongoing human review, to measure error rates and catch novel patterns.

B. Re-running the original validation set every quarter and confirming the accuracy is unchanged.

C. Lowering the confidence threshold every month by a small fixed amount to keep up with new formats.

D. Reviewing only low-confidence extractions, since they are where the model has already signaled uncertainty.

End of examination. Stop here, then check your answers using the key that follows.

5. Quick answer key and score tracker

Mark each question you answered correctly (all required selections, no wrong ones). Then complete the domain tracker below.

No.	Ans.	Domain	Scen.	Mine
1	**B**	D1	S1	
2	**C**	D1	S1	
3	**A**	D1	S1	
4	**D**	D1	S1	
5	**B**	D1	S1	
6	**C**	D1	S1	
7	**C**	D1	S1	
8	**B**	D2	S1	
9	**D**	D2	S1	
10	**D**	D4	S1	
11	**B**	D5	S1	
12	**B**	D5	S1	
13	**D**	D5	S1	
14	**D**	D5	S1	
15	**C**	D5	S1	
16	**A**	D1	S2	
17	**B**	D1	S2	
18	**B,C,D**	D1	S2	
19	**D**	D2	S2	
20	**A**	D2	S2	
21	**D**	D2	S2	
22	**D**	D3	S2	
23	**C**	D3	S2	
24	**A**	D3	S2	
25	**D**	D3	S2	
26	**C**	D3	S2	
27	**B**	D3	S2	
28	**C**	D3	S2	
29	**C**	D3	S2	
30	**B**	D5	S2	

No.	Ans.	Domain	Scen.	Mine
31	**D**	D5	S2	
32	**A**	D1	S5	
33	**C**	D1	S5	
34	**B,D**	D1	S5	
35	**D**	D2	S5	
36	**D**	D2	S5	
37	**B**	D3	S5	
38	**D,E**	D3	S5	
39	**A**	D3	S5	
40	**A**	D3	S5	
41	**D**	D4	S5	
42	**A**	D4	S5	
43	**A**	D4	S5	
44	**A,E**	D4	S5	
45	**C**	D4	S5	
46	**D,E**	D1	S6	
47	**B**	D1	S6	
48	**A**	D1	S6	
49	**A**	D2	S6	
50	**A,B**	D2	S6	
51	**C**	D2	S6	
52	**C**	D2	S6	
53	**B**	D4	S6	
54	**A**	D4	S6	
55	**B**	D4	S6	
56	**B**	D4	S6	
57	**B**	D4	S6	
58	**A**	D4	S6	
59	**C**	D5	S6	
60	**A**	D5	S6	

Domain-level score tracker

Domain	Questions	Number list	My correct	% correct
1. Agentic Architecture & Orchestration	16	1, 2, 3, 4, 5, 6, 7, 16, 17, 18, 32, 33, 34, 46, 47, 48		
2. Tool Design & MCP Integration	11	8, 9, 19, 20, 21, 35, 36, 49, 50, 51, 52		
3. Claude Code Configuration & Workflows	12	22, 23, 24, 25, 26, 27, 28, 29, 37, 38, 39, 40		
4. Prompt Engineering & Structured Output	12	10, 41, 42, 43, 44, 45, 53, 54, 55, 56, 57, 58		
5. Context Management & Reliability	9	11, 12, 13, 14, 15, 30, 31, 59, 60		
Total	**60**			

Interpretation: percent-correct here is a practice signal only. The real exam's pass/fail decision is based on a single scaled score (cut score 720 of 1,000); domain percentages on the real score report are informational.

6. Detailed answer key and explanations

Each entry gives the correct answer, why it is correct, why every other option is wrong, the concept tested, the detail in the scenario that decides the answer, and a production lesson.

Q1 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.5 | Intermediate | Single answer

Correct answer: B - A pre-call hook on process_refund that blocks amounts above $500 and redirects the case to the escalate_to_human workflow with the case details.

Why the correct answer is correct: Hooks that intercept outgoing tool calls enforce compliance rules deterministically, for example blocking refunds above a threshold and redirecting to an alternative workflow such as human escalation. The business rule is an exact numeric limit, so code should apply it rather than the model.

Option-by-option:

A. **[INCORRECT]** A stronger system prompt that repeats the $500 limit in capital letters at the start and end of the instructions and adds two examples of correct behavior.
Emphasis and examples reduce but do not eliminate violations. A prompt-based rule keeps a non-zero failure rate, which is unacceptable for a financial limit.

B. **[CORRECT]** A pre-call hook on process_refund that blocks amounts above $500 and redirects the case to the escalate_to_human workflow with the case details.
Correct. The hook enforces the threshold deterministically and keeps the customer moving by redirecting to the right workflow.

C. **[INCORRECT]** A nightly report listing refunds above $500 so a manager can reverse any that should not have been issued autonomously.
Detection after the fact leaves the money already paid out and relies on a reversal that may not be possible or timely.

D. **[INCORRECT]** A requirement that the agent self-report its confidence before each refund and skip any refund where confidence is below 90 percent.
Confidence does not track the dollar amount, and self-reported confidence is poorly calibrated. The rule is about amount, not certainty.

Key concept: Tool call interception hooks versus prompt-based enforcement (Task 1.5, 1.4).
Exam reasoning: A residual 3% violation rate under a clear prompt shows the instruction is being followed probabilistically. A hard financial threshold needs a guarantee.
Production lesson: When a rule can be expressed as a check on tool arguments, enforce it in code and give the agent a safe alternative path.

Q2 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.1 | Challenging | Single answer

Correct answer: C - Execute every tool_use block in the response and return a matching tool_result for each tool_use ID in the next message, then continue the loop.

Why the correct answer is correct: When stop_reason is 'tool_use', the response may contain several tool_use blocks. The loop must run all of them and append tool_result blocks that correspond to each tool_use ID so the model sees the outcome of every request before deciding the next step.

Option-by-option:

A. **[INCORRECT]** Treat a response with more than one tool_use block as an error and ask the model to retry with a single tool call.
Parallel tool requests are valid. Rejecting them wastes a round trip and discards a useful capability.

B. **[INCORRECT]** Combine both results into a single tool_result block tied to the first tool_use ID and ignore the second ID.
Every tool_use ID needs its own matching tool_result, so merging them leaves the second request unanswered.

C. **[CORRECT]** Execute every tool_use block in the response and return a matching tool_result for each tool_use ID in the next message, then continue the loop.
Correct. Each tool request needs its own result so the model can reason about all of them.

D. **[INCORRECT]** Execute only the first tool_use block and discard the second, since the model will request it again on the next turn if the result is still needed.
The unanswered tool_use block leaves the history inconsistent, and the model does not simply re-ask.

Key concept: Agentic loop mechanics and tool results in history (Task 1.1).
Exam reasoning: Two independent tool requests in one response and a loop that handles only one explains the odd behavior.

Production lesson: Treat tool calls as a list, not a single value. Pair each request with its result.

Q3 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.5 | Foundational | Single answer

Correct answer: A - A PostToolUse hook that masks card numbers down to the last four digits in tool results before the model processes them.

Why the correct answer is correct: PostToolUse hooks intercept tool results before the model processes them. Masking sensitive fields there applies least privilege to data: the model gets what it needs (last four digits) and nothing more.

Option-by-option:

A. **[CORRECT]** A PostToolUse hook that masks card numbers down to the last four digits in tool results before the model processes them.
Correct. The model receives only the masked value, so it cannot repeat what it never saw.

B. **[INCORRECT]** A system prompt instruction telling the agent never to write a full card number in any reply and to mask digits when it does mention a card.
The model still receives the sensitive value and can leak it when the prompt rule fails.

C. **[INCORRECT]** Remove lookup_order from the agent's tools and have staff paste payment details into the chat when needed.
This breaks the workflow and moves sensitive data handling to humans without need.

D. **[INCORRECT]** A post-processing filter on the final customer message that removes any 16-digit sequence before the reply is sent.
This helps at the output boundary but the sensitive data is still in the model's context, logs, and any subagent handoffs.

Key concept: PostToolUse hooks for transformation and data minimization (Task 1.5).
Exam reasoning: The requirement that the model never see full numbers rules out approaches that act only after the model has the data.
Production lesson: Minimize what enters the context. Data the model never sees cannot be leaked by it.

Q4 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.6 | Foundational | Single answer

Correct answer: D - A fixed sequential prompt chain with one focused step per aspect, each step receiving the ticket and the previous findings as needed.

Why the correct answer is correct: Prompt chaining suits predictable multi-aspect reviews where the steps are known ahead of time. Dynamic decomposition suits open-ended tasks where subtasks depend on what is discovered.

Option-by-option:

A. **[INCORRECT]** A dynamic plan where the model decides at runtime which aspects to check for each ticket and in what order.
Adaptive planning suits open-ended investigation. Here it would make the audit inconsistent between tickets.

B. **[INCORRECT]** Four independent agents with their own tools that never share findings and a coordinator that picks the one with the longest report.
Independent agents add coordination cost, and selecting by report length is not a quality criterion.

C. **[INCORRECT]** A single request asking for a complete audit of all four aspects in one response, with a list of criteria at the end.
One broad pass tends to give uneven depth across aspects.

D. **[CORRECT]** A fixed sequential prompt chain with one focused step per aspect, each step receiving the ticket and the previous findings as needed.
Correct. The aspects are known in advance, so a predictable chain gives consistent, auditable results.

Key concept: Choosing between prompt chaining and dynamic decomposition (Task 1.6).
Exam reasoning: The same four aspects for every ticket mean the work is predictable, which favors a fixed chain.
Production lesson: Use the least dynamic structure that fits the task.

Q5 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.4 | Foundational | Single answer

Correct answer: B - Have the tool layer attach an idempotency key derived from the order and request, so repeated calls for the same refund return the original result instead of paying again.

Why the correct answer is correct: A timeout does not tell you whether the side effect happened. Making the operation idempotent, with a stable key per intended refund, lets the system retry safely. Guarantees for financial actions belong in the tool and its wrapper, not in model discipline.

Option-by-option:

A. **[INCORRECT]** Remove the retry logic so that any timeout is reported to the customer as a failure and left for a human to resolve.
This avoids duplicates but turns every slow response into an unresolved case, hurting first-contact resolution.

B. **[CORRECT]** Have the tool layer attach an idempotency key derived from the order and request, so repeated calls for the same refund return the original result instead of paying again.
Correct. Idempotency makes a retry safe regardless of whether the first attempt succeeded.

C. **[INCORRECT]** Add a system prompt rule telling the agent to call lookup_order before every retry to check whether the refund was already issued, and to skip the retry if it finds a matching refund.
The agent may skip the check, and there is still a race between the check and the retry.

D. **[INCORRECT]** Lower the timeout so the wrapper gives up faster and reports failures sooner.
A shorter timeout increases the number of ambiguous outcomes. It does not make retries safe.

Key concept: Deterministic safeguards against duplicate side effects (Task 1.4, 1.5).
Exam reasoning: The first call succeeded despite the timeout, which is the classic ambiguous-outcome case that idempotency addresses.
Production lesson: Every retried write needs an idempotency story.

Q6 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.6 | Challenging | Single answer, secondary: D2, D5

Correct answer: C - Handle the concerns as separate items, answer A and B with their results, and clearly tell the customer that C could not be checked yet, offering a retry or escalation.

Why the correct answer is correct: Multi-concern requests should be decomposed, with each concern investigated and the results synthesized. When one lookup fails, local recovery comes first, and anything unresolved is reported honestly alongside the valid results rather than hidden or allowed to block everything.

Option-by-option:

A. **[INCORRECT]** Reply only about the order that succeeded first and ignore the other two to keep the answer short.
Ignoring concerns leaves the request unresolved.

B. **[INCORRECT]** Hold the entire reply until the lookup for order C succeeds, retrying silently in the background, so that the customer receives one complete and consistent answer at the end of the conversation.
Silent retries can stall the whole response and the customer receives nothing meanwhile.

C. **[CORRECT]** Handle the concerns as separate items, answer A and B with their results, and clearly tell the customer that C could not be checked yet, offering a retry or escalation.
Correct. Partial results are delivered and the failure is stated rather than hidden.

D. **[INCORRECT]** Report that all three lookups failed so the answer is consistent, and ask the customer to try again later.
That misstates what happened and discards the valid results.

Key concept: Decomposition with partial failure handling (Task 1.6, 2.2, 5.3).
Exam reasoning: Two of three succeeded, so a good design preserves their value and is transparent about the third.
Production lesson: Return what you know, say what you don't, and offer a next step.

Q7 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.5 | Intermediate | Single answer

Correct answer: C - Refunds over $500 must be routed to a human instead of being issued by the agent.

Why the correct answer is correct: Hooks provide deterministic guarantees where business rules require compliance. Prompt instructions are appropriate for soft guidance such as tone and tool preferences, where occasional deviation is tolerable.

Option-by-option:

A. **[INCORRECT]** Replies should sound warm and empathetic while staying concise.
Tone is a judgment-based style preference, which belongs in the prompt. A hook cannot meaningfully check it.

B. **[INCORRECT]** The agent should summarize what it did at the end of each conversation.
A closing summary is a soft behavior expectation. A missed summary has low cost.

C. **[CORRECT]** Refunds over $500 must be routed to a human instead of being issued by the agent.
Correct. It is a hard, checkable rule with financial consequences, which needs deterministic enforcement.

D. **[INCORRECT]** When an order number is mentioned, the agent should generally prefer lookup_order over get_customer.
This is tool-selection guidance, better handled by tool descriptions and examples.

Key concept: Choosing hooks versus prompts (Task 1.5).
Exam reasoning: Only the refund threshold is both objectively checkable and costly if violated.
Production lesson: Use code for rules and prompts for judgment.

Q8 | Domain 2: Tool Design & MCP Integration | Scenario 1: Customer Support Resolution Agent

Task statement 2.2 | Challenging | Single answer

Correct answer: B - Return an error result with category transient, isRetryable true, and a description, reserving an empty successful result for a lookup that ran and found nothing.

Why the correct answer is correct: Access failures need retry decisions, while valid empty results are successful queries with no matches. Returning structured errors with category and retryability prevents the agent from converting an outage into a false statement.

Option-by-option:

A. **[INCORRECT]** Keep returning an empty list in both cases, but add an optional free-text note field that the service fills in on some errors to explain to the agent what went wrong.
Overloading the empty success hides the distinction unless the model reliably reads an optional note.

B. **[CORRECT]** Return an error result with category transient, isRetryable true, and a description, reserving an empty successful result for a lookup that ran and found nothing.
Correct. The agent can tell an outage from a real empty history and can retry or apologize accordingly.

C. **[INCORRECT]** Return the HTTP status code number as the tool output and let the model interpret it.
Raw status codes are not a stable contract and do not convey retryability or a customer-friendly description.

D. **[INCORRECT]** Have the agent call the tool three times and treat three empty results as proof the customer has no orders.
Repeated calls to a down service return the same misleading result.

Key concept: Access failures versus valid empty results (Task 2.2).
Exam reasoning: The incorrect statement 'you have no orders' is what happens when failure is reported as emptiness.
Production lesson: Make failure impossible to confuse with absence.

Q9 | Domain 2: Tool Design & MCP Integration | Scenario 1: Customer Support Resolution Agent

Task statement 2.4 | Intermediate | Single answer

Correct answer: D - Use an existing community MCP server for the standard ticketing integration and build a custom server only for the refund logic, which is specific to your workflows.

Why the correct answer is correct: Existing community MCP servers are preferable for standard integrations such as Jira or ticketing, while custom servers are reserved for team-specific workflows.

Option-by-option:

A. **[INCORRECT]** Skip MCP and have the agent call each system through Bash with curl commands that developers write by hand.
This gives up structured tool interfaces and increases shell-command risk.

B. **[INCORRECT]** Build custom servers for both integrations so that every tool description is written in the team's own style and wording, no outside code is trusted, and the team controls every update.
Consistent style does not justify maintaining a custom copy of a standard integration.

C. **[INCORRECT]** Use community servers for both and encode the refund approval rules as a long section in the system prompt.
Approval rules are business logic that needs enforcement in your own tool or hook, not in a prompt.

D. **[CORRECT]** Use an existing community MCP server for the standard ticketing integration and build a custom server only for the refund logic, which is specific to your workflows.
Correct. Reuse for standard integrations, custom only where the logic is yours.

Key concept: Choosing community versus custom MCP servers (Task 2.4).
Exam reasoning: One integration is generic and one is company-specific, so the answer splits along that line.
Production lesson: Build what differentiates you. Reuse what doesn't.

Q10 | Domain 4: Prompt Engineering & Structured Output | Scenario 1: Customer Support Resolution Agent

Task statement 4.1 | Intermediate | Single answer

Correct answer: D - Define each priority with explicit criteria and a concrete example case (for example amount thresholds, account standing, and time sensitivity) so classification is consistent.

Why the correct answer is correct: Explicit severity criteria with concrete examples for each level achieve consistent classification. Vague definitions like 'important and urgent' leave the boundary to chance.

Option-by-option:

A. **[INCORRECT]** Ask for a numeric confidence score alongside each priority and use it to resolve disagreements between runs.
Confidence does not define what each priority means.

B. **[INCORRECT]** Run each dispute through the classifier five times and use the most common label.
Voting hides an undefined standard and multiplies cost.

C. **[INCORRECT]** Ask the agent to think carefully about urgency, to compare each new dispute with how it labeled earlier disputes, and to keep its priority labels consistent from one dispute to the next.
Comparing with earlier labels needs memory the classifier may not have, and the criteria stay vague.

D. **[CORRECT]** Define each priority with explicit criteria and a concrete example case (for example amount thresholds, account standing, and time sensitivity) so classification is consistent.
Correct. Explicit severity criteria with concrete examples produce consistent classification.

Key concept: Explicit severity criteria (Task 4.1).
Exam reasoning: Different labels for the same dispute show the criteria are underspecified.
Production lesson: Define categories so two reviewers, human or model, would agree.

Q11 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.2 | Intermediate | Single answer

Correct answer: B - Escalate to a human, because the policy is silent on competitor price matching, and include the facts of the request in the handoff.

Why the correct answer is correct: Escalation triggers include policy exceptions or gaps, not just complex cases. When policy is ambiguous or silent on the specific request, escalate rather than guessing.

Option-by-option:

A. **[INCORRECT]** Approve the match under the own-site price adjustment rule since it is the closest rule available.
Stretching a rule beyond what it says creates an unauthorized exception.

B. **[CORRECT]** Escalate to a human, because the policy is silent on competitor price matching, and include the facts of the request in the handoff.
Correct. A gap in policy is a legitimate escalation trigger.

C. **[INCORRECT]** Offer a goodwill credit for a modest amount to settle the conversation and avoid involving others.
Inventing compensation outside policy is also an unauthorized decision.

D. **[INCORRECT]** Decline the request because it is not covered, and close the case so the customer can look for other options.
Declining may be wrong if the business would allow an exception, and the decision is not the agent's to make.

Key concept: Escalating on policy gaps (Task 5.2).
Exam reasoning: The policy covers own-site adjustments only, so competitor matching is a gap.
Production lesson: Agents should not invent policy. They should surface its gaps.

Q12 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.2 | Intermediate | Single answer

Correct answer: B - Ask the customer for an additional identifier, such as the email address or postal code on the account, before proceeding.

Why the correct answer is correct: When tool results return multiple matches, the agent should ask for additional identifiers rather than choosing by heuristics.

Option-by-option:

A. **[INCORRECT]** Proceed with the first account returned and mention to the customer which one was used.
Informing the customer afterwards does not prevent an incorrect action on the wrong account.

B. **[CORRECT]** Ask the customer for an additional identifier, such as the email address or postal code on the account, before proceeding.
Correct. Multiple matches require clarification instead of a guess.

C. **[INCORRECT]** Select the account with the most recent order, since that is the most likely one the customer means, and proceed with the refund on it right away.
Heuristic selection can target the wrong person and expose or refund the wrong account.

D. **[INCORRECT]** Escalate immediately to a human because a common name makes the case too ambiguous to handle.
A quick clarifying question usually resolves it, and escalating wastes the opportunity to resolve in first contact.

Key concept: Ambiguity resolution with multiple matches (Task 5.2).
Exam reasoning: Acting on the wrong account has financial and privacy consequences, so a clarifying question is the cheapest safe step.
Production lesson: Ask before acting when identity is ambiguous.

Q13 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.2 | Intermediate | Single answer

Correct answer: D - Acknowledge the frustration, offer the replacement or refund the policy allows, and escalate only if the customer then insists on a human.

Why the correct answer is correct: The agent should acknowledge frustration while offering resolution when the issue is within its capability, escalating only if the customer reiterates a preference for a human. Sentiment and confidence scores are unreliable proxies.

Option-by-option:

A. **[INCORRECT]** Escalate immediately because strong negative sentiment indicates the case is too sensitive for an automated agent to handle without a person involved.
Sentiment is not a reliable proxy for case complexity, and escalating a resolvable issue lowers first-contact resolution.

B. **[INCORRECT]** Ask the customer to rate their frustration from 1 to 10 and escalate if the number is above 7.
A self-rated scale adds friction and does not measure whether the agent can solve the problem.

C. **[INCORRECT]** Ignore the tone and respond only with the order status so the exchange stays factual.
Ignoring frustration is poor service and may make the customer insist on a human.

D. **[CORRECT]** Acknowledge the frustration, offer the replacement or refund the policy allows, and escalate only if the customer then insists on a human.
Correct. The issue is within the agent's capability, and escalation follows an actual request.

Key concept: Escalation versus resolution under frustration (Task 5.2).
Exam reasoning: No human was requested and the fix is within policy, which makes resolution the right first move.
Production lesson: Distinguish a request for a person from an expression of emotion.

Q14 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.1 | Intermediate | Single answer

Correct answer: D - Put a short summary of the key findings at the beginning and organize the detailed results under explicit section headers.

Why the correct answer is correct: Models tend to process the beginning and end of long inputs more reliably than the middle. Placing key findings first and using explicit section headers for details mitigates this lost-in-the-middle effect.

Option-by-option:

A. **[INCORRECT]** Increase the amount of detail in every section so the important finding is repeated several times throughout.
Padding makes the input longer and does not guarantee the model uses the key finding.

B. **[INCORRECT]** Ask the final agent to re-read the combined text ten times before writing the response.
Repetition inside one turn does not change how attention is distributed.

C. **[INCORRECT]** Append the three results in random order each run so that no investigation is systematically placed in the middle.
Randomizing hides the pattern but does not fix the omissions.

D. **[CORRECT]** Put a short summary of the key findings at the beginning and organize the detailed results under explicit section headers.
Correct. This mitigates position effects in long inputs.

Key concept: Position-aware input ordering (Task 5.1).
Exam reasoning: The omitted finding sat in the middle of a long combined input, the classic position problem.
Production lesson: Structure long inputs as summary first, details labelled after.

Q15 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.1 | Challenging | Single answer

Correct answer: C - Change the investigators to return structured data, such as key facts, citations, and relevance scores, instead of long reasoning narratives.

Why the correct answer is correct: When downstream agents have limited context budgets, upstream agents should return structured data (key facts, citations, relevance scores) instead of verbose content and reasoning chains.

Option-by-option:

A. **[INCORRECT]** Increase the coordinator's context by moving to a larger-window model, and keep the narratives unchanged so that no reasoning detail is thrown away.
More room delays the problem and does not improve signal density or attention.

B. **[INCORRECT]** Have the coordinator truncate each narrative to its first 500 characters before reading it.
Truncation may cut off the facts that matter, which are not necessarily at the start.

C. **[CORRECT]** Change the investigators to return structured data, such as key facts, citations, and relevance scores, instead of long reasoning narratives.
Correct. Upstream agents should return compact structured results when downstream context is limited.

D. **[INCORRECT]** Add a summarizer subagent that rewrites every narrative into a paragraph before the coordinator sees it.
That adds another lossy step when the investigators could produce structured output directly.

Key concept: Structured upstream outputs for limited context (Task 5.1).
Exam reasoning: The coordinator needs only a few fields, so verbose narratives waste its budget.
Production lesson: Design agent outputs for their consumer, not for the producer's convenience.

Q16 | Domain 1: Agentic Architecture & Orchestration | Scenario 2: Code Generation with Claude Code

Task statement 1.7 | Foundational | Single answer

Correct answer: A - Resume the named session with --resume payment-timeout and continue from where you stopped.

Why the correct answer is correct: Named session resumption with --resume <session-name> continues a specific prior conversation. It is appropriate when the prior context is mostly valid.

Option-by-option:

A. **[CORRECT]** Resume the named session with --resume payment-timeout and continue from where you stopped.
Correct. Named sessions are designed to be resumed across work sessions.

B. **[INCORRECT]** Run /init so Claude regenerates its understanding of the repository from scratch.
/init sets up project memory. It does not restore a prior conversation.

C. **[INCORRECT]** Create a new CLAUDE.md note that says 'continue the payment-timeout investigation' and start a new session.
A one-line note carries none of the findings from the previous session.

D. **[INCORRECT]** Start a fresh session and paste the terminal scrollback from Monday so Claude can reconstruct the context.
Pasting scrollback is noisy, loses structure, and wastes context.

Key concept: Named session resumption (Task 1.7).
Exam reasoning: The requirement is the exact investigation with accumulated context, which is what resuming a named session provides.
Production lesson: Name long-running investigations so they can be resumed deliberately.

Q17 | Domain 1: Agentic Architecture & Orchestration | Scenario 2: Code Generation with Claude Code

Task statement 1.6 | Intermediate | Single answer

Correct answer: B - Use an adaptive investigation: map the system, form hypotheses from the evidence, and generate the next subtasks based on what each step reveals.

Why the correct answer is correct: Dynamic adaptive decomposition fits open-ended investigation: subtasks are generated based on what is discovered at each step. Fixed pipelines suit predictable work.

Option-by-option:

A. **[INCORRECT]** Split the checklist into four parallel agents and merge their reports without further investigation.
Parallelism does not help if the checklist itself omits the cause.

B. **[CORRECT]** Use an adaptive investigation: map the system, form hypotheses from the evidence, and generate the next subtasks based on what each step reveals.
Correct. Open-ended investigations need decomposition that adapts to findings.

C. **[INCORRECT]** Keep the fixed checklist but extend it to 40 items covering every plausible dependency, and have Claude work through the items strictly in order each time.
A longer fixed list still cannot anticipate an unknown cause and wastes effort on irrelevant items.

D. **[INCORRECT]** Ask Claude to propose a complete root cause immediately from the logs and then verify it at the end.
Committing early to a cause invites confirmation bias.

Key concept: Adaptive decomposition for open-ended tasks (Task 1.6).
Exam reasoning: The cause was outside the checklist previously, which signals that discovery must drive the plan.
Production lesson: Don't script what you don't yet understand.

Q18 | Domain 1: Agentic Architecture & Orchestration | Scenario 2: Code Generation with Claude Code

Task statement 1.7 | Intermediate | Select 3

Correct answer: B, C, D - After code changes, a resumed session should be told which files changed so it can re-analyze them rather than re-explore everything. / When most earlier tool results are stale, starting a new session with a structured summary is more reliable than resuming. / fork_session can create independent branches from a shared analysis baseline to compare two refactoring approaches.

Why the correct answer is correct: Session management involves choosing between resumption (when prior context is mostly valid) and a new session with injected summary (when tool results are stale), telling resumed sessions about specific file changes, and using fork_session for divergent exploration from a shared baseline.

Option-by-option:

A. **[INCORRECT]** Resuming is always more reliable than starting fresh because the full history is preserved.
A full history includes stale results, which can make resumption less reliable.

B. **[CORRECT]** After code changes, a resumed session should be told which files changed so it can re-analyze them rather than re-explore everything.
Correct. Targeted re-analysis is efficient and avoids reasoning from stale file state.

C. **[CORRECT]** When most earlier tool results are stale, starting a new session with a structured summary is more reliable than resuming.
Correct. Stale results can mislead the model and a summary carries only what is still valid.

D. **[CORRECT]** fork_session can create independent branches from a shared analysis baseline to compare two refactoring approaches.
Correct. Forks let you explore divergent approaches from the same starting point.

E. **[INCORRECT]** fork_session creates an isolated copy of the repository so the branches can never affect each other's files.
Forking branches the conversation. It is not a repository copy.

F. **[INCORRECT]** A resumed session automatically re-reads any file that has changed on disk since the session was last used.
The session does not refresh its earlier tool results on its own, so it must be told or must re-read.

Key concept: Resumption, fresh start, and forking (Task 1.7).
Exam reasoning: Each correct statement matches a documented practice, and each incorrect one assumes automatic refresh or isolation that does not exist.
Production lesson: Treat session state as a conversation, not as a snapshot of the filesystem.

Q19 | Domain 2: Tool Design & MCP Integration | Scenario 2: Code Generation with Claude Code

Task statement 2.5 | Intermediate | Single answer

Correct answer: D - Grep for the import path pattern across the codebase and keep the matches in test files.

Why the correct answer is correct: Grep searches file contents (imports, function names, error messages) and Glob matches path patterns. The task asks about contents, so Grep is the efficient first move.

Option-by-option:

A. **[INCORRECT]** Glob for **/billing/utils* to find all files with that name and report them.
Glob matches file paths, so it lists the module itself rather than files that import it.

B. **[INCORRECT]** Use Bash with find and cat to print every file in the repo and search the output manually.
This floods the context and bypasses the dedicated search tools.

C. **[INCORRECT]** Glob for **/*.test.* and then Read every matching test file to look for the import.
This reads a large number of files, many irrelevant, to answer a content question.

D. **[CORRECT]** Grep for the import path pattern across the codebase and keep the matches in test files.
Correct. The question is about file contents, which Grep searches directly.

Key concept: Selecting Grep versus Glob (Task 2.5).
Exam reasoning: 'Import from' is a content pattern, not a filename pattern.
Production lesson: Match the tool to what you are searching: names with Glob, content with Grep.

Q20 | Domain 2: Tool Design & MCP Integration | Scenario 2: Code Generation with Claude Code

Task statement 2.5 | Intermediate | Single answer, secondary: D5

Correct answer: A - Grep for the discount entry point, Read that file, then follow its imports to trace the flow step by step.

Why the correct answer is correct: Build codebase understanding incrementally: start with Grep to find entry points, then use Read to follow imports and trace flows, rather than reading all files upfront.

Option-by-option:

A. **[CORRECT]** Grep for the discount entry point, Read that file, then follow its imports to trace the flow step by step.
Correct. Incremental exploration keeps context focused.

B. **[INCORRECT]** Read every file under the checkout directory first so that Claude holds the whole module in context before answering.
Reading everything up front consumes context and buries the relevant code.

C. **[INCORRECT]** Ask Claude to explain how checkouts typically apply discounts in similar applications and rely on that description.
Generic patterns may not match this code.

D. **[INCORRECT]** Glob for all files with 'discount' in the name and treat that list as the complete implementation.
Logic may live in files whose names do not include the word.

Key concept: Incremental code exploration (Task 2.5, 5.4).
Exam reasoning: A large unfamiliar repository makes reading everything impractical.
Production lesson: Follow the trail, don't read the forest.

Q21 | Domain 2: Tool Design & MCP Integration | Scenario 2: Code Generation with Claude Code

Task statement 2.4 | Challenging | Single answer

Correct answer: D - Whether GITHUB_TOKEN is set in the environment of the shell that launches Claude Code, since the config expands that variable at startup.

Why the correct answer is correct: Project-scoped .mcp.json can use environment variable expansion such as ${GITHUB_TOKEN} so secrets are not committed. If the variable is missing in a developer's environment, the server cannot authenticate, which makes an environment-specific failure the first thing to check.

Option-by-option:

A. **[INCORRECT]** Whether the teammate should reinstall Claude Code because the project-level configuration is only read by fresh installs.
Project configuration is read regardless of how recently the tool was installed.

B. **[INCORRECT]** Whether the GitHub server definition should be moved into CLAUDE.md so Claude Code reads it as an instruction.
CLAUDE.md holds instructions, not server definitions.

C. **[INCORRECT]** Whether the token should be pasted directly into .mcp.json for that teammate and committed so the config matches everyone's.
Committing a literal token exposes the secret and is not needed for one person's environment.

D. **[CORRECT]** Whether GITHUB_TOKEN is set in the environment of the shell that launches Claude Code, since the config expands that variable at startup.
Correct. A missing variable is the most likely reason for a failed server with identical config.

Key concept: MCP configuration and connection troubleshooting (Task 2.4).
Exam reasoning: Same repository and different outcome points to something outside the repository: the environment.
Production lesson: When a shared configuration fails for one person, check what is not shared: their environment.

Q22 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.1 | Intermediate | Single answer

Correct answer: D - In a CLAUDE.md inside the payments/ directory, committed to the repository.

Why the correct answer is correct: CLAUDE.md can exist at user, project, and directory levels. A directory-level file carries conventions for that subtree and is shared through version control.

Option-by-option:

A. **[INCORRECT]** In each developer's ~/.claude/CLAUDE.md so the rules are always loaded whichever project they open.
User-level files are personal, not shared, and would apply to unrelated projects.

B. **[INCORRECT]** In the root CLAUDE.md so that every instruction exists in a single file loaded everywhere.
This loads payment-only rules for all work, which adds noise.

C. **[INCORRECT]** In a custom command that developers run before editing anything in payments/.
A command depends on people remembering to run it.

D. **[CORRECT]** In a CLAUDE.md inside the payments/ directory, committed to the repository.
Correct. Directory-level CLAUDE.md files scope conventions to that area and are shared via version control.

Key concept: CLAUDE.md hierarchy and directory scope (Task 3.1).
Exam reasoning: The rules apply to one directory and must be shared, which points to a committed directory-level file.
Production lesson: Place instructions at the narrowest scope where they apply.

Q23 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.5 | Intermediate | Single answer

Correct answer: C - Share all three failures together in a single detailed message so Claude can address the interacting cause at once.

Why the correct answer is correct: When fixes interact, provide all issues in a single message. When problems are independent, sequential iteration works well.

Option-by-option:

A. **[INCORRECT]** Report only the first failure and wait to see whether the others disappear, ignoring them until then.
That may work by luck, but it delays discovering the interaction.

B. **[INCORRECT]** Fix them one at a time in three separate sessions, so each fix starts with a clean context and cannot be influenced by the other two.
Separate sessions lose the shared understanding that the problems are related.

C. **[CORRECT]** Share all three failures together in a single detailed message so Claude can address the interacting cause at once.
Correct. When problems interact, one comprehensive message avoids fixes that undo each other.

D. **[INCORRECT]** Ask Claude to disable the three failing tests until the cache work is complete.
Disabling tests removes the verification you need.

Key concept: Single versus sequential issue feedback (Task 3.5).
Exam reasoning: Fixing one changes the others, which is the sign of interacting issues.
Production lesson: Give the model the whole picture when the problems are one problem.

Q24 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.3 | Challenging | Single answer

Correct answer: A - Use a recursive glob such as src/api/**/* so files in nested directories match as well.

Why the correct answer is correct: Path-specific rules use YAML frontmatter paths with glob patterns. A recursive pattern like src/api/**/* matches files in subdirectories, whereas a single-level pattern only matches direct children.

Option-by-option:

A. **[CORRECT]** Use a recursive glob such as src/api/**/* so files in nested directories match as well.
*Correct. A single * does not cross directory boundaries, while ** matches nested paths.*

B. **[INCORRECT]** Remove the paths field entirely so the rule loads on every request regardless of the file being edited.
This makes the rule unconditional, losing the point of path scoping and adding context everywhere.

C. **[INCORRECT]** Move the conventions into a skill and ask developers to invoke it for deeper directories.
Skills are invoked on demand, so the conventions would no longer apply automatically.

D. **[INCORRECT]** Add a separate rules file for each nested directory and list the same conventions in every one of them.
This duplicates content and breaks again when new subdirectories appear.

Key concept: Glob patterns in path-scoped rules (Task 3.3).
Exam reasoning: The rule works at the top level but not in nested folders, which points to the pattern depth.
Production lesson: Test glob patterns against your deepest paths, and check with /memory what has loaded.

Q25 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.1 | Challenging | Single answer

Correct answer: D - Run /memory in both sessions to see which memory files are loaded, then compare, for example to find a rule that exists only in one developer's user-level file.

Why the correct answer is correct: The /memory command verifies which memory files are loaded and helps diagnose inconsistent behavior across sessions or users, such as an instruction living only in a user-level file.

Option-by-option:

A. **[INCORRECT]** Ask both developers to reinstall Claude Code and clone the repository again from scratch, before anything else, to rule out a corrupted installation or checkout.
A reinstall does not reveal which instructions are loaded, and the cause is likely configuration, not corruption.

B. **[INCORRECT]** Compare both developers' Claude Code versions and downgrade the newer one.
Version differences are possible, but this is not the first, cheapest diagnostic step.

C. **[INCORRECT]** Add the convention three more times to the project CLAUDE.md so it is certain to be noticed.
Repeating an instruction does not fix a loading difference and clutters the file.

D. **[CORRECT]** Run /memory in both sessions to see which memory files are loaded, then compare, for example to find a rule that exists only in one developer's user-level file.
Correct. /memory shows the loaded files, making the difference visible.

Key concept: Using /memory to diagnose configuration (Task 3.1).
Exam reasoning: Same repository and different behavior suggests different loaded context, which /memory reveals directly.
Production lesson: Observe before you change. Check what actually loaded.

Q26 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.3 | Foundational | Single answer

Correct answer: C - The rules load only when working on matching files, which keeps irrelevant instructions out of context and reduces token usage.

Why the correct answer is correct: Path-scoped rules load only when editing matching files, reducing irrelevant context and token usage.

Option-by-option:

A. **[INCORRECT]** Rules files can be invoked by typing a slash command, which makes them easier to find.
Rules load by file path, and slash commands are a different mechanism.

B. **[INCORRECT]** Rules files take priority over CLAUDE.md instructions whenever the two disagree about the same topic, which makes them safer for strict conventions.
Path scoping is about when content loads, not about overriding precedence.

C. **[CORRECT]** The rules load only when working on matching files, which keeps irrelevant instructions out of context and reduces token usage.
Correct. Conditional loading saves context for unrelated work.

D. **[INCORRECT]** Rules files run faster than CLAUDE.md because they are compiled into a binary format.
No such compilation exists and speed is not the reason.

Key concept: Benefit of conditional rules (Task 3.3).
Exam reasoning: The Terraform conventions are relevant to only one part of the repository.
Production lesson: Load context only when it is relevant.

Q27 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.2 | Foundational | Single answer

Correct answer: B - Add an argument-hint to the skill's frontmatter, such as [ticket-id], so developers are shown the required parameter when they invoke the skill.

Why the correct answer is correct: argument-hint frontmatter prompts developers for required parameters when they invoke the skill without arguments.

Option-by-option:

A. **[INCORRECT]** Add a sentence to CLAUDE.md saying every skill must be invoked with all required parameters.
A general reminder is easy to ignore and doesn't tell the user what the parameter is.

B. **[CORRECT]** Add an argument-hint to the skill's frontmatter, such as [ticket-id], so developers are shown the required parameter when they invoke the skill.
Correct. argument-hint tells the developer what to supply when invoking the skill.

C. **[INCORRECT]** Add context: fork to the skill's frontmatter so that it runs in a sub-agent, which can then ask the developer for the missing ticket ID.
context: fork isolates execution and output. It does not collect missing arguments.

D. **[INCORRECT]** Move the skill to ~/.claude/skills/ so developers each see the prompt privately.
Moving the file changes who has it, not what the invocation prompts for.

Key concept: Skill frontmatter: argument-hint (Task 3.2).
Exam reasoning: The issue is a missing parameter at invocation, which argument-hint addresses.
Production lesson: Put usage guidance where it appears at invocation.

Q28 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.4 | Challenging | Single answer

Correct answer: C - X direct execution, Y plan mode, Z direct execution.

Why the correct answer is correct: Plan mode suits tasks with architectural implications or multiple valid approaches. Direct execution suits simple, well-scoped changes such as single-method fixes or one-file renames.

Option-by-option:

A. **[INCORRECT]** X direct execution, Y direct execution, Z direct execution, adjusting if complications appear.
Y's infrastructure choice should be made deliberately before building, not discovered mid-implementation.

B. **[INCORRECT]** X plan mode, Y plan mode, Z direct execution.
Planning for a clearly scoped crash fix adds overhead without benefit.

C. **[CORRECT]** X direct execution, Y plan mode, Z direct execution.
Correct. Only Y has architectural trade-offs and multiple valid approaches.

D. **[INCORRECT]** X plan mode, Y plan mode, Z plan mode, so every change follows the same process.
Uniform process wastes effort on trivial changes.

Key concept: Plan mode versus direct execution (Task 3.4).
Exam reasoning: Y involves different infrastructure requirements; X and Z have clear scope.
Production lesson: Spend planning effort in proportion to uncertainty and blast radius.

Q29 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.5 | Intermediate | Single answer

Correct answer: C - Write the tests first covering expected behavior, edge cases, and performance, then have Claude implement and iterate by sharing test failures.

Why the correct answer is correct: Test-driven iteration, writing test suites first covering behavior, edge cases, and performance requirements, then iterating by sharing failures, guides progressive improvement.

Option-by-option:

A. **[INCORRECT]** Describe the requirement in more detail and ask Claude to say 'done' only when it is confident it is correct.
Self-assessed confidence does not detect missed edge cases.

B. **[INCORRECT]** Generate three independent implementations and keep the one with the fewest lines.
Brevity is not correctness.

C. **[CORRECT]** Write the tests first covering expected behavior, edge cases, and performance, then have Claude implement and iterate by sharing test failures.
Correct. Tests make the requirements concrete and failures give targeted feedback.

D. **[INCORRECT]** Ask Claude to write the implementation first and then write tests that match whatever the implementation does, so that the tests are guaranteed to pass.
Tests derived from the implementation confirm its behavior rather than the requirements.

Key concept: Test-driven iteration (Task 3.5).
Exam reasoning: Previous prose attempts missed edge cases, which tests make explicit.
Production lesson: Executable requirements beat descriptive ones.

Q30 | Domain 5: Context Management & Reliability | Scenario 2: Code Generation with Claude Code

Task statement 5.4 | Challenging | Single answer, secondary: D3

Correct answer: B - Delegate each question to a subagent that explores in isolation and returns a concise summary, keeping the main session for coordination.

Why the correct answer is correct: Subagent delegation isolates verbose exploration output while the main agent keeps high-level understanding, and summaries from subagents are injected into context instead of the raw discovery.

Option-by-option:

A. **[INCORRECT]** Run both investigations directly in the main session so the details stay in view, and use /compact only when the context actually overflows its limit.
Verbose output would push the session toward overflow and degrade reasoning before it is compacted.

B. **[CORRECT]** Delegate each question to a subagent that explores in isolation and returns a concise summary, keeping the main session for coordination.
Correct. Subagent delegation isolates verbose exploration output.

C. **[INCORRECT]** Ask Claude to skip tracing and refactor based on its general knowledge of billing systems.
Skipping investigation risks incorrect changes to unfamiliar code.

D. **[INCORRECT]** Clear the conversation and re-explain the refactor goals from memory, then run the investigations.
Clearing discards the findings and decisions made so far.

Key concept: Subagent delegation for exploration (Task 5.4, 3.4).
Exam reasoning: The main session is already heavily used and still needs to coordinate, which favors isolating new exploration.
Production lesson: Keep the coordinator's context for decisions, not for raw discovery.

Q31 | Domain 5: Context Management & Reliability | Scenario 2: Code Generation with Claude Code

Task statement 5.4 | Intermediate | Single answer

Correct answer: D - Use /compact to reduce context usage, and keep referring to the notes file for the specifics.

Why the correct answer is correct: Use /compact to reduce context usage during extended exploration when context fills with verbose discovery output; scratchpad files persist key findings across context boundaries.

Option-by-option:

A. **[INCORRECT]** Keep working until the session errors, then start over without the notes file.
Waiting for failure wastes the time of degraded answers and discards the notes.

B. **[INCORRECT]** Open a second terminal and run the same session in both windows so the load is shared.
Duplicating sessions doesn't reduce context use.

C. **[INCORRECT]** Delete the notes file so Claude must rediscover the findings and therefore pays closer attention.
Deleting notes removes the durable state and wastes effort.

D. **[CORRECT]** Use /compact to reduce context usage, and keep referring to the notes file for the specifics.
Correct. Compaction reclaims space and the notes keep the key findings durable.

Key concept: /compact and scratchpad files (Task 5.4).
Exam reasoning: Notes already preserve the specifics, so compaction is low-risk.
Production lesson: Compact context, but keep durable facts outside it.

Q32 | Domain 1: Agentic Architecture & Orchestration | Scenario 5: Claude Code for Continuous Integration

Task statement 1.6 | Foundational | Single answer

Correct answer: A - A fixed sequential prompt chain for the nightly report, and dynamic adaptive decomposition for the latency investigation.

Why the correct answer is correct: Prompt chaining suits predictable multi-aspect reviews, while dynamic decomposition suits open-ended investigations whose subtasks depend on what is discovered.

Option-by-option:

A. **[CORRECT]** A fixed sequential prompt chain for the nightly report, and dynamic adaptive decomposition for the latency investigation.
Correct. Predictable work gets a chain; open-ended investigation gets an adaptive plan.

B. **[INCORRECT]** Dynamic adaptive decomposition for the nightly report, and a fixed prompt chain for the latency investigation.
This reverses the fit: the report becomes inconsistent and the investigation cannot adapt.

C. **[INCORRECT]** Dynamic adaptive decomposition for both jobs so each run is tailored to the repository.
Per-run variation undermines the repeatable nightly report.

D. **[INCORRECT]** A fixed prompt chain for both jobs so behavior is predictable and easy to maintain across the platform.
A fixed chain cannot follow unexpected findings in the latency investigation.

Key concept: Matching decomposition to workflow (Task 1.6).
Exam reasoning: One job has known stages and one has an unknown cause.
Production lesson: Choose structure by predictability.

Q33 | Domain 1: Agentic Architecture & Orchestration | Scenario 5: Claude Code for Continuous Integration

Task statement 1.6 | Challenging | Single answer, secondary: D4

Correct answer: C - The structured per-file findings plus a compact description of changed interfaces and data flows, with the full code of only the files involved in each cross-file concern.

Why the correct answer is correct: Large reviews are split into per-file local analysis passes plus a separate integration pass. The integration pass should work from compact, structured results and targeted code so it can reason about cross-file data flow without attention dilution.

Option-by-option:

A. **[INCORRECT]** The full text of all 12 files again, plus a longer prompt asking it to examine every file as carefully as the local passes did, in a single pass.
This recreates the attention dilution that the split was meant to avoid.

B. **[INCORRECT]** Only the list of file names, so that the integration pass is not biased by the local findings.
File names alone provide no basis for analyzing data flow.

C. **[CORRECT]** The structured per-file findings plus a compact description of changed interfaces and data flows, with the full code of only the files involved in each cross-file concern.
Correct. It focuses on relationships without re-diluting attention across everything.

D. **[INCORRECT]** Nothing at all, running the integration pass on the PR title to keep it independent of earlier passes.
Without content it cannot find any cross-file issue.

Key concept: Multi-pass review design (Task 1.6, 4.6).
Exam reasoning: Shallow integration output after giving it everything again shows the integration pass needs a focused input.
Production lesson: Pass distilled results upward, not raw material again.

Q34 | Domain 1: Agentic Architecture & Orchestration | Scenario 5: Claude Code for Continuous Integration

Task statement 1.6 | Intermediate | Select 2

Correct answer: B, D - Adding tests to a poorly understood legacy module, by mapping the structure, then prioritizing and adapting the plan as dependencies appear. / Reviewing every PR for the same fixed list of concerns, using a prompt chain with one step per concern.

Why the correct answer is correct: Prompt chaining suits predictable work; dynamic decomposition suits open-ended tasks; large reviews should be split into local and integration passes.

Option-by-option:

A. **[INCORRECT]** Generating the same weekly summary format for every team, using a fully dynamic plan that the model invents on each run.
A repeated, fixed deliverable should be predictable, not reinvented each time.

B. **[CORRECT]** Adding tests to a poorly understood legacy module, by mapping the structure, then prioritizing and adapting the plan as dependencies appear.
Correct. Open-ended work benefits from adaptive decomposition.

C. **[INCORRECT]** Reviewing a 14-file PR, using one pass over all files with a prompt asking for equal depth on each.
A single pass over many files risks uneven depth. Per-file plus integration passes work better.

D. **[CORRECT]** Reviewing every PR for the same fixed list of concerns, using a prompt chain with one step per concern.
Correct. Known stages make a chain appropriate.

E. **[INCORRECT]** Diagnosing an unfamiliar production incident, using a fixed five-step script written before the incident.
An unknown cause cannot be investigated with a script written in advance.

Key concept: Selecting decomposition patterns (Task 1.6).
Exam reasoning: The two correct pairings each match structure to predictability; the others invert or ignore it.
Production lesson: Ask whether the steps are known before choosing the pattern.

Q35 | Domain 2: Tool Design & MCP Integration | Scenario 5: Claude Code for Continuous Integration

Task statement 2.2 | Intermediate | Single answer

Correct answer: D - Return an MCP error with category permission, isRetryable false, and a description naming the missing scope, so the agent stops and reports the cause.

Why the correct answer is correct: Structured error metadata distinguishes transient, validation, business, and permission errors and signals retryability, so agents avoid wasted retries and can report accurately.

Option-by-option:

A. **[INCORRECT]** Return the same 'Request failed' message but include a timestamp so engineers can find the matching server log.
Logs help humans afterward, but the agent still cannot distinguish a permission error from a transient one.

B. **[INCORRECT]** Return an empty metadata object so the review proceeds without metadata and appears to succeed.
Hiding the failure produces a review based on missing information.

C. **[INCORRECT]** Add a system prompt instruction that the agent should retry any failing tool call at most five times.
A retry cap bounds the waste but still retries something that can never succeed.

D. **[CORRECT]** Return an MCP error with category permission, isRetryable false, and a description naming the missing scope, so the agent stops and reports the cause.
Correct. The agent can avoid futile retries and the pipeline fails with a clear, fixable message.

Key concept: Permission errors and retryability (Task 2.2).
Exam reasoning: Five futile retries of a permission failure is the symptom of an undifferentiated error.
Production lesson: Tell the agent whether trying again can help.

Q36 | Domain 2: Tool Design & MCP Integration | Scenario 5: Claude Code for Continuous Integration

Task statement 2.2 | Intermediate | Single answer, secondary: D5

Correct answer: D - Return an error indicating the search did not complete, with the failure type and whether retrying is advised, instead of an empty success.

Why the correct answer is correct: Access failures and valid empty results are different. An incomplete query must be reported as an error so the agent does not draw conclusions from absence of results.

Option-by-option:

A. **[INCORRECT]** Return an empty result in both cases, but add a warning sentence to the system prompt explaining that the tool can be rate limited and may return nothing.
The model cannot tell from the tool output whether a given empty result was real.

B. **[INCORRECT]** Lower the reviewer's authority by labeling every deletion suggestion as advisory, regardless of the evidence.
This reduces the harm of mistakes but doesn't give correct information.

C. **[INCORRECT]** Make the reviewer run every search twice and accept the deletion only if both runs are empty.
Two rate-limited calls can both be empty.

D. **[CORRECT]** Return an error indicating the search did not complete, with the failure type and whether retrying is advised, instead of an empty success.
Correct. A failed search must not look like a search with no matches.

Key concept: Failure versus empty success (Task 2.2, 5.3).
Exam reasoning: The incorrect conclusion came from reading a failure as 'no matches'.
Production lesson: Absence of evidence is only evidence when the search ran.

Q37 | Domain 3: Claude Code Configuration & Workflows | Scenario 5: Claude Code for Continuous Integration

Task statement 3.6 | Foundational | Single answer

Correct answer: B - In the project's CLAUDE.md, which provides project context to Claude Code including when it is invoked from CI.

Why the correct answer is correct: CLAUDE.md provides project context (testing standards, fixture conventions, review criteria) to Claude Code, including CI-invoked runs. Documenting valuable-test criteria and fixtures improves generation quality.

Option-by-option:

A. **[INCORRECT]** In a comment at the top of each test file, since Claude only reads the file it is currently editing.
Scattered comments are hard to maintain and are not the project-wide mechanism.

B. **[CORRECT]** In the project's CLAUDE.md, which provides project context to Claude Code including when it is invoked from CI.
Correct. CLAUDE.md is the mechanism for project standards and fixture conventions.

C. **[INCORRECT]** In a developer's ~/.claude/CLAUDE.md, because CI picks up the personal file of whoever triggered the build.
User-level files are personal and aren't part of the shared repository.

D. **[INCORRECT]** In the pipeline's job log, so that the output of earlier runs is visible to future runs automatically.
Logs are output, not input, and aren't loaded as instructions.

Key concept: CLAUDE.md for CI context (Task 3.6).
Exam reasoning: The standards must be shared with CI, which has no developer home directory.
Production lesson: Put team knowledge where every invocation, human or CI, will find it.

Q38 | Domain 3: Claude Code Configuration & Workflows | Scenario 5: Claude Code for Continuous Integration

Task statement 3.6 | Challenging | Select 2

Correct answer: D, E - Provide the existing test files in context so test generation avoids suggesting scenarios already covered. / Include the prior review findings in the context and instruct Claude to report only new or still-unaddressed issues.

Why the correct answer is correct: When re-running reviews after new commits, include prior findings and instruct Claude to report only new or unaddressed issues; provide existing test files so test generation avoids duplicates.

Option-by-option:

A. **[INCORRECT]** Run the review only on the final commit and disable it on every earlier commit.
This delays feedback and doesn't address the duplicate test suggestions.

B. **[INCORRECT]** Delete all earlier review comments before every re-run so the PR starts clean each time.
This removes the history developers rely on and doesn't stop the model from repeating issues.

C. **[INCORRECT]** Raise the sampling temperature so repeated runs naturally produce different wording and different suggestions.
Randomness changes phrasing, not whether the issue was already reported.

D. **[CORRECT]** Provide the existing test files in context so test generation avoids suggesting scenarios already covered.
Correct. Visibility into current tests prevents redundant suggestions.

E. **[CORRECT]** Include the prior review findings in the context and instruct Claude to report only new or still-unaddressed issues.
Correct. This avoids duplicate comments on later commits.

Key concept: Incremental CI review and test generation (Task 3.6).
Exam reasoning: Both complaints are about missing knowledge of what already exists.
Production lesson: Give CI runs memory of earlier output.

Q39 | Domain 3: Claude Code Configuration & Workflows | Scenario 5: Claude Code for Continuous Integration

Task statement 3.6 | Intermediate | Single answer, secondary: D4

Correct answer: A - Run Claude Code with --output-format json and a --json-schema describing the findings so output is machine-parseable.

Why the correct answer is correct: The --output-format json and --json-schema flags enforce structured output in CI, yielding machine-parseable findings for automated posting as inline comments.

Option-by-option:

A. **[CORRECT]** Run Claude Code with --output-format json and a --json-schema describing the findings so output is machine-parseable.
Correct. Schema-constrained output gives stable fields for automated posting.

B. **[INCORRECT]** Ask Claude to output the findings twice in different formats and use whichever one parses successfully.
Doubling output wastes tokens and still depends on free-form text.

C. **[INCORRECT]** Switch the pipeline to interactive mode so a person can copy comments into the PR by hand.
Interactive mode hangs automation and defeats the purpose of CI.

D. **[INCORRECT]** Add stricter instructions in the prompt about exactly how to format headings and bullets, and keep the regular expressions.

Formatting instructions help but still produce drift that breaks brittle parsers.

Key concept: Structured CI output (Task 3.6, 4.3).
Exam reasoning: Parser breakage from format drift is solved by a contract, not by better prose.
Production lesson: Parse structure, never prose.

Q40 | Domain 3: Claude Code Configuration & Workflows | Scenario 5: Claude Code for Continuous Integration

Task statement 3.6 | Intermediate | Single answer, secondary: D4

Correct answer: A - Run the review in a separate, independent Claude Code invocation that has not seen the generation reasoning, receiving only the diff and project context.

Why the correct answer is correct: A session that generated code is less effective at reviewing it because it retains the reasoning context. An independent review instance is more effective at catching subtle issues.

Option-by-option:

A. **[CORRECT]** Run the review in a separate, independent Claude Code invocation that has not seen the generation reasoning, receiving only the diff and project context.
Correct. An independent instance is more likely to question the code.

B. **[INCORRECT]** Skip review for changes Claude generates because they were already written with care.
Generated code needs review at least as much as human-written code.

C. **[INCORRECT]** Keep the same session and tell Claude to act as a strict external reviewer that has never seen the code before, using all the project's review criteria.
Role instructions do not remove the retained reasoning context from generation.

D. **[INCORRECT]** Ask the same session to review three times and take the most critical of the three reviews.
Repeating self-review within the same context doesn't make it independent.

Key concept: Session context isolation (Task 3.6, 4.6).
Exam reasoning: Consistent approval of flawed work is the signature of self-review bias.
Production lesson: Separate the author from the reviewer.

Q41 | Domain 4: Prompt Engineering & Structured Output | Scenario 5: Claude Code for Continuous Integration

Task statement 4.4 | Foundational | Single answer

Correct answer: D - Retry with a follow-up that includes the original diff, the failed output, and the specific validation error, asking Claude to correct it.

Why the correct answer is correct: Retry-with-error-feedback appends specific validation errors to the prompt, which works for format and structural errors. It is not effective when the required information is absent from the source.

Option-by-option:

A. **[INCORRECT]** Retry the original request unchanged up to ten times and accept the first valid response.
Blind retries ignore the information the validator provided.

B. **[INCORRECT]** Fill the missing field with a default value of 'low' in code so that the pipeline always receives a valid object and the job continues without interruption.
Guessing a severity hides information and may understate real problems.

C. **[INCORRECT]** Fail the CI job on the first validation error, since invalid output means the model is unreliable.
A single fixable format error shouldn't invalidate the whole run.

D. **[CORRECT]** Retry with a follow-up that includes the original diff, the failed output, and the specific validation error, asking Claude to correct it.
Correct. Specific feedback on a structural error usually fixes it.

Key concept: Validation-retry loops (Task 4.4).
Exam reasoning: The information exists and only the structure is wrong, so targeted feedback should succeed.
Production lesson: Tell the model exactly what failed.

Q42 | Domain 4: Prompt Engineering & Structured Output | Scenario 5: Claude Code for Continuous Integration

Task statement 4.3 | Intermediate | Single answer

Correct answer: A - A programmatic check that each finding's file and line fall within the diff hunks, with invalid findings rejected or sent back for correction.

Why the correct answer is correct: Strict schemas via tool use eliminate syntax errors but do not prevent semantic errors such as values in wrong fields or out-of-range references. Semantic checks must be explicit.

Option-by-option:

A. **[CORRECT]** A programmatic check that each finding's file and line fall within the diff hunks, with invalid findings rejected or sent back for correction.
Correct. Schema validity does not guarantee semantic correctness, so check semantics in code.

B. **[INCORRECT]** A stricter JSON schema that makes the line number field required, an integer, and greater than zero, so that obviously invalid numbers cannot be returned.
Even a required positive integer can still point outside the diff; the problem is its value, which the schema cannot judge.

C. **[INCORRECT]** Switching tool_choice from 'any' to 'auto' so the model can decide when to output findings.
This changes whether a tool is called, not the correctness of the values.

D. **[INCORRECT]** A prompt addition reminding the model that line numbers must be correct and carefully checked.
A reminder does not provide a check and the error may recur.

Key concept: Schema validity versus semantic correctness (Task 4.3, 4.4).
Exam reasoning: The JSON is valid yet wrong, which is the exact case schemas cannot cover.
Production lesson: Validate meaning separately from shape.

Q43 | Domain 4: Prompt Engineering & Structured Output | Scenario 5: Claude Code for Continuous Integration

Task statement 4.5 | Intermediate | Single answer

Correct answer: A - Use the Message Batches API for the weekly audit, but gather the needed cross-file context before submission, because a batch request cannot call tools mid-request.

Why the correct answer is correct: The Message Batches API offers 50% savings with up to a 24-hour window and no latency guarantee, and it does not support multi-turn tool calling within a single request. It suits non-blocking, latency-tolerant work.

Option-by-option:

A. **[CORRECT]** Use the Message Batches API for the weekly audit, but gather the needed cross-file context before submission, because a batch request cannot call tools mid-request.
Correct. The workload tolerates latency, but batches don't support multi-turn tool calling within a request.

B. **[INCORRECT]** Avoid batches because the audit needs tools, and run it synchronously to avoid any restrictions.
A latency-tolerant weekly job is a good fit for batching if context is prepared up front.

C. **[INCORRECT]** Use the Message Batches API and let each request call Grep during processing as needed, since tool use works the same way in batches and returns results mid-request.
Batch requests cannot execute tools mid-request and return results.

D. **[INCORRECT]** Use batches for pre-merge checks as well, since 50% savings apply to any workload that calls the API.
Pre-merge checks block developers, so unpredictable batch latency is unacceptable.

Key concept: Batch API suitability and limits (Task 4.5).
Exam reasoning: The weekend schedule fits batch processing, and the mid-request tool need is the limitation to design around.
Production lesson: Know both when batch fits and what it cannot do.

Q44 | Domain 4: Prompt Engineering & Structured Output | Scenario 5: Claude Code for Continuous Integration

Task statement 4.3 | Challenging | Select 2

Correct answer: A, E - Make suggested_fix nullable so that the model can return null when there is no concrete fix, instead of inventing one. / Use an enum for category with an 'other' option and a free-text detail field for unlisted types.

Why the correct answer is correct: Schemas should make fields optional or nullable when the source may not contain the information, and use enums with 'other' plus detail fields for extensible categories. tool_choice 'auto' does not guarantee structured output.

Option-by-option:

A. **[CORRECT]** Make suggested_fix nullable so that the model can return null when there is no concrete fix, instead of inventing one.
Correct. Optional or nullable fields prevent fabrication to satisfy required fields.

B. **[INCORRECT]** Use a free-form string for severity so the model can describe it in its own words.
Free text makes severity inconsistent and hard to route.

C. **[INCORRECT]** Make every field required so the model must always provide complete information for each finding.
Required fields for missing information push the model to fabricate values.

D. **[INCORRECT]** Rely on tool_choice 'auto' to guarantee that findings always come back through the schema.
With 'auto', the model may answer with text instead of calling the tool.

E. **[CORRECT]** Use an enum for category with an 'other' option and a free-text detail field for unlisted types.
Correct. It keeps categories extensible without forcing wrong labels.

Key concept: Schema design for structured output (Task 4.3).
Exam reasoning: Each correct choice prevents a specific failure: fabrication, or forced mislabeling.
Production lesson: Design schemas that allow honest answers.

Q45 | Domain 4: Prompt Engineering & Structured Output | Scenario 5: Claude Code for Continuous Integration

Task statement 4.5 | Challenging | Single answer

Correct answer: C - Every 4 hours, because a document waits at most 4 hours for the next submission plus up to 24 hours of processing.

Why the correct answer is correct: Worst-case latency equals the wait for the next submission plus the maximum processing window. To guarantee 30 hours with a 24-hour window, the submission interval must be 6 hours or less; 4 hours satisfies this.

Option-by-option:

A. **[INCORRECT]** Every 12 hours, which keeps costs low by creating fewer, larger batches.
12 + 24 = 36 hours breaks the SLA.

B. **[INCORRECT]** Every 24 hours, matching the maximum processing window so the schedule is easy to remember.
24 + 24 = 48 hours is far outside the SLA.

C. **[CORRECT]** Every 4 hours, because a document waits at most 4 hours for the next submission plus up to 24 hours of processing.
Correct. 4 + 24 = 28 hours, which is within 30.

D. **[INCORRECT]** Every 8 hours, because a document waits at most 8 hours and batches usually finish well before 24 hours.
8 + 24 = 32 hours exceeds the SLA in the worst case, and 'usually' is not a guarantee.

Key concept: Batch submission frequency and SLA calculation (Task 4.5).
Exam reasoning: The guarantee depends on worst case, not typical case.
Production lesson: Compute SLAs from upper bounds.

Q46 | Domain 1: Agentic Architecture & Orchestration | Scenario 6: Structured Data Extraction

Task statement 1.5 | Intermediate | Select 2

Correct answer: D, E - A pre-call hook on write_to_ledger that blocks the call unless the record has passed schema and business-rule validation. / A PostToolUse hook that normalizes OCR dates into a single ISO format before the model processes the result.

Why the correct answer is correct: PostToolUse hooks transform tool results before the model sees them, and outgoing-call hooks can block actions until prerequisites are satisfied.

Option-by-option:

A. **[INCORRECT]** A system prompt paragraph asking the agent to remember to validate each record before writing it.
That is prompt-based guidance, not a hook, and it has a non-zero failure rate.

B. **[INCORRECT]** A hook that increases max_tokens whenever the OCR output is long so the model can reason more.
Token limits do not address inconsistent formats or unvalidated writes.

C. **[INCORRECT]** A hook that rewrites the model's extracted vendor names to what it believes they should be, based on fuzzy matching.
Silently changing extracted values hides errors and is a data-quality decision that needs review, not a hook.

D. **[CORRECT]** A pre-call hook on write_to_ledger that blocks the call unless the record has passed schema and business-rule validation.
Correct. It deterministically stops unvalidated records from reaching the ledger.

E. **[CORRECT]** A PostToolUse hook that normalizes OCR dates into a single ISO format before the model processes the result.
Correct. Normalizing at the boundary gives the model consistent data.

Key concept: Hooks for normalization and gating (Task 1.5).
Exam reasoning: One need is consistent input and the other is guaranteed gating of an outgoing write.
Production lesson: Put deterministic transformations and gates in code around the model.

Q47 | Domain 1: Agentic Architecture & Orchestration | Scenario 6: Structured Data Extraction

Task statement 1.5 | Foundational | Single answer

Correct answer: B - A tool-call interception hook that checks the total on write_to_ledger and, above the threshold, redirects the record to the approval queue.

Why the correct answer is correct: Hooks intercepting outgoing tool calls enforce compliance rules and can redirect to alternative workflows. When business rules require guaranteed compliance, choose hooks over prompts.

Option-by-option:

A. **[INCORRECT]** A line in the system prompt stating clearly that invoices with a total over $10,000 must never be posted and must go to the approval queue instead.
A prompt rule has a non-zero failure rate and the consequence is a financial posting.

B. **[CORRECT]** A tool-call interception hook that checks the total on write_to_ledger and, above the threshold, redirects the record to the approval queue.
Correct. The check runs on every outgoing call and cannot be skipped by the model.

C. **[INCORRECT]** A few-shot example showing a large invoice being sent to the queue, added to the extraction prompt.
Examples improve behavior but don't guarantee it.

D. **[INCORRECT]** A daily report of ledger entries above $10,000 for finance to review the next morning.
Review after posting doesn't prevent the posting.

Key concept: Threshold enforcement with redirect (Task 1.5).
Exam reasoning: 'Must never' signals a rule that needs deterministic enforcement.
Production lesson: Make irreversible actions unreachable when the rule says so.

Q48 | Domain 1: Agentic Architecture & Orchestration | Scenario 6: Structured Data Extraction

Task statement 1.6 | Intermediate | Single answer

Correct answer: A - Extract obligations section by section in focused passes, then run a separate reconciliation pass that checks cross-section definitions and conflicts.

Why the correct answer is correct: Splitting large inputs into per-section local passes plus a separate integration pass avoids attention dilution and handles cross-section issues.

Option-by-option:

A. **[CORRECT]** Extract obligations section by section in focused passes, then run a separate reconciliation pass that checks cross-section definitions and conflicts.
Correct. Local passes maintain depth and the integration pass handles cross-section relationships.

B. **[INCORRECT]** Use only the first 40 pages, where extraction quality is highest, and mark the remaining pages as out of scope.
This silently drops most of the contract.

C. **[INCORRECT]** Run the same single request three times and merge the obligations that appear in at least two runs.
Consensus filtering may remove real obligations found in only some runs and doesn't fix coverage of later sections.

D. **[INCORRECT]** Keep the single request but ask the model to give every section equal attention and to double-check later sections.
An instruction does not change attention dilution across a very long input.

Key concept: Prompt chaining for long documents (Task 1.6).
Exam reasoning: Uneven depth and cross-section contradictions are the two problems the two-stage design addresses.
Production lesson: Break long work into focused passes, then reconcile.

Q49 | Domain 2: Tool Design & MCP Integration | Scenario 6: Structured Data Extraction

Task statement 2.3 | Intermediate | Single answer, secondary: D4

Correct answer: A - Set tool_choice to 'any' so Claude must call one of the extraction tools, while still choosing which one fits the document.

Why the correct answer is correct: tool_choice 'auto' allows the model to return text; 'any' requires a tool call but lets the model choose which; forced selection names a specific tool. When the document type is unknown and several schemas exist, 'any' guarantees structured output.

Option-by-option:

A. **[CORRECT]** Set tool_choice to 'any' so Claude must call one of the extraction tools, while still choosing which one fits the document.
Correct. 'any' guarantees a tool call and leaves tool selection to the model.

B. **[INCORRECT]** Force the extract_invoice tool for every document, regardless of type, so that the pipeline always receives structured data in one predictable schema.
Forcing one tool would mis-extract receipts and contracts.

C. **[INCORRECT]** Remove two of the three tools and use one generic tool that accepts any document type.
This loses document-specific schemas and may not solve the missing tool call.

D. **[INCORRECT]** Keep tool_choice 'auto' and add a sentence to the prompt asking Claude never to answer with text.
With 'auto' the model may still respond with text, and a prompt request is not a guarantee.

Key concept: tool_choice behavior (Task 2.3, 4.3).
Exam reasoning: Unknown document types rule out forcing one tool, and the text replies rule out 'auto'.
Production lesson: Match tool_choice to what you know about the input.

Q50 | Domain 2: Tool Design & MCP Integration | Scenario 6: Structured Data Extraction

Task statement 2.1 | Foundational | Select 2

Correct answer: A, B - Split the overlapping responsibilities into purpose-specific tools (for example extract_invoice_fields and parse_layout_to_text), each with a defined input and output contract. / Rename the tools to purpose-specific names and rewrite their descriptions with inputs, outputs, and when to use each.

Why the correct answer is correct: Tool descriptions are the primary mechanism for selection. Overlap is resolved by renaming, rewriting descriptions, or splitting generic tools into purpose-specific ones with clear contracts.

Option-by-option:

A. **[CORRECT]** Split the overlapping responsibilities into purpose-specific tools (for example extract_invoice_fields and parse_layout_to_text), each with a defined input and output contract.
Correct. Narrow tools with clear input/output contracts are easier to select reliably.

B. **[CORRECT]** Rename the tools to purpose-specific names and rewrite their descriptions with inputs, outputs, and when to use each.
Correct. Clear names and descriptions eliminate functional overlap.

C. **[INCORRECT]** Add a third tool called analyze_everything that wraps both so the model has a safe default.
A catch-all adds more overlap.

D. **[INCORRECT]** Reduce both descriptions to a single word so the model relies on tool names alone.
Minimal descriptions are the original cause of unreliable selection.

E. **[INCORRECT]** Instruct the model in the system prompt to alternate between the two tools to even out usage.
Alternation has nothing to do with the document or the purpose.

Key concept: Eliminating tool overlap (Task 2.1).
Exam reasoning: Interchangeable use of near-identical tools is the textbook overlap problem.
Production lesson: Make each tool's purpose unmistakable.

Q51 | Domain 2: Tool Design & MCP Integration | Scenario 6: Structured Data Extraction

Task statement 2.3 | Challenging | Single answer, secondary: D4

Correct answer: C - Force extract_metadata with tool_choice on the first request, then continue in follow-up turns once its result is available.

Why the correct answer is correct: Use tool_choice forced selection ({"type": "tool", "name": "extract_metadata"}) to ensure a particular tool is called first, then process subsequent steps in follow-up turns.

Option-by-option:

A. **[INCORRECT]** Force extract_metadata on every request in the conversation so it always runs before anything else.
Forcing it every turn prevents the enrichment tools from ever being called.

B. **[INCORRECT]** Set tool_choice to 'any' on the first request so the model must call a tool, and trust it to pick the right one.
'any' requires a tool call but not that specific tool.

C. **[CORRECT]** Force extract_metadata with tool_choice on the first request, then continue in follow-up turns once its result is available.
Correct. Forced selection guarantees the first call, and later turns can proceed normally.

D. **[INCORRECT]** Describe the required order in the system prompt and list extract_metadata first in the tools array.
Prompt order and list position are not enforcement mechanisms.

Key concept: Forced tool selection for ordering (Task 2.3, 4.3).
Exam reasoning: The requirement is a guaranteed first step, which forced selection delivers; forcing every turn would break later steps.
Production lesson: Force only the step that must happen, and only when it must happen.

Q52 | Domain 2: Tool Design & MCP Integration | Scenario 6: Structured Data Extraction

Task statement 2.1 | Intermediate | Single answer

Correct answer: C - Expand the descriptions with input formats, example documents, edge cases such as credit notes, and boundaries stating when to use each tool.

Why the correct answer is correct: Tool descriptions should include input formats, example queries, edge cases, and boundary explanations so the model can select among similar tools.

Option-by-option:

A. **[INCORRECT]** Merge the two tools into one extract_financial_document tool with a type enum and a one-line description.
The model still gets no guidance on what counts as an invoice versus a receipt, so the same edge cases are misrouted inside the merged tool.

B. **[INCORRECT]** Tell the agent in the prompt to prefer extract_invoice whenever the document contains any amount.
That would send genuine receipts to the wrong tool.

C. **[CORRECT]** Expand the descriptions with input formats, example documents, edge cases such as credit notes, and boundaries stating when to use each tool.
Correct. Edge cases and boundaries in descriptions guide selection for ambiguous inputs.

D. **[INCORRECT]** Add a regular expression that looks for the phrase 'credit note' and switches the tool call in application code after the model chooses.
A patch for a single phrase doesn't generalize to similar documents and bypasses the model.

Key concept: Tool description boundaries and edge cases (Task 2.1).
Exam reasoning: The misrouting is of a specific edge case, which descriptions can name explicitly.
Production lesson: Put known edge cases into the tool contract.

Q53 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.4 | Intermediate | Single answer

Correct answer: B - Make po_number nullable and route invoices without it to review, instead of retrying, since the information is absent from the provided source.

Why the correct answer is correct: Retries are ineffective when required information is absent from the source. Fields should be optional or nullable when documents may not contain the information, to prevent fabrication.

Option-by-option:

A. **[INCORRECT]** Add a regular expression check that rejects PO numbers that don't match the format, and retry on mismatch.
Fabricated values can match the format and the information is still missing.

B. **[CORRECT]** Make po_number nullable and route invoices without it to review, instead of retrying, since the information is absent from the provided source.
Correct. Retries cannot recover information that is not in the source, and a required field encourages fabrication.

C. **[INCORRECT]** Add more retries with the validation error attached each time, for every invoice, until the model finally supplies a PO number that passes the format check.
More retries will produce more fabricated values.

D. **[INCORRECT]** Ask the model to use the invoice number as the PO number whenever it cannot find one.
That systematically records wrong data.

Key concept: Limits of retry and nullable fields (Task 4.4, 4.3).
Exam reasoning: The PO number lives in an external document that the model does not have.

Production lesson: Don't ask a model to find what isn't there.

Q54 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.5 | Intermediate | Single answer

Correct answer: A - Identify the failures by custom_id, split those 7 documents into chunks, and resubmit only them.

Why the correct answer is correct: Batch failures are handled by resubmitting only the failed documents identified by custom_id with appropriate modifications, such as chunking documents that exceeded context limits.

Option-by-option:

A. **[CORRECT]** Identify the failures by custom_id, split those 7 documents into chunks, and resubmit only them.
Correct. custom_id correlates failures, and chunking addresses the cause.

B. **[INCORRECT]** Discard the 7 failed documents because batch processing is best effort.
Dropping data silently loses records.

C. **[INCORRECT]** Resubmit the entire batch of 100 unchanged so the results are consistent.
This repeats the failures and pays again for the 93 successes.

D. **[INCORRECT]** Move all 100 documents to the synchronous API without changes to handle failures faster.
The oversized documents would still exceed the context limit.

Key concept: Batch failure handling (Task 4.5).
Exam reasoning: The cause is document size, so the resubmission must change the input.
Production lesson: Retry what failed, and fix what caused it.

Q55 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.4 | Intermediate | Single answer

Correct answer: B - Extract the stated total as printed and also a calculated total from the line items, and flag any discrepancy for review.

Why the correct answer is correct: Self-correction validation flows extract a calculated_total alongside the stated_total to flag discrepancies. Schema syntax is guaranteed but semantic checks need explicit design.

Option-by-option:

A. **[INCORRECT]** Ask the model to double-check that its numbers add up before it responds.
A request is not a verification and the model can miss it.

B. **[CORRECT]** Extract the stated total as printed and also a calculated total from the line items, and flag any discrepancy for review.
Correct. Comparing two values detects semantic inconsistency that schemas cannot.

C. **[INCORRECT]** Replace the stated total with the sum of line items in all cases, as the sum is always more reliable.
The document may be right and the line items incomplete, so overriding hides information.

D. **[INCORRECT]** Make the total field required and numeric, and add a minimum value of zero, so that the model cannot omit it or return something invalid.
The field is already valid and the discrepancy remains.

Key concept: Semantic validation and discrepancy flags (Task 4.4).
Exam reasoning: The failure is a mismatch between values, so comparing values is the detection method.
Production lesson: Capture both numbers and let code compare them.

Q56 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.3 | Foundational | Single answer

Correct answer: B - Strict schemas via tool use eliminate JSON syntax errors, but values can still be semantically wrong, for example in the wrong field or inconsistent with each other.

Why the correct answer is correct: Tool use with JSON schemas is the most reliable way to obtain schema-compliant output and eliminates syntax errors, but it does not prevent semantic errors.

Option-by-option:

A. **[INCORRECT]** Strict schemas guarantee both valid structure and correct values as long as every field is marked required.
Required fields do not make the values correct and can encourage fabrication.

B. **[CORRECT]** Strict schemas via tool use eliminate JSON syntax errors, but values can still be semantically wrong, for example in the wrong field or inconsistent with each other.
Correct. Shape is guaranteed and meaning is not.

C. **[INCORRECT]** Strict schemas only apply to the first request in a conversation and are ignored in later turns.
Nothing in tool use limits schema enforcement to the first turn.

D. **[INCORRECT]** Strict schemas help only when the prompt contains few-shot examples; otherwise the output may not be valid JSON.
Schema enforcement via tool use does not depend on examples for validity.

Key concept: What schemas do and do not guarantee (Task 4.3).
Exam reasoning: The teammate conflates syntactic validity with correctness.
Production lesson: Validate both shape and meaning.

Q57 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.4 | Intermediate | Single answer, secondary: D5

Correct answer: B - Include a conflict_detected flag, record both values with their locations, and route the record to review instead of silently choosing one.

Why the correct answer is correct: Adding conflict_detected booleans and capturing inconsistent values enables downstream handling of contradictory sources instead of arbitrary selection.

Option-by-option:

A. **[INCORRECT]** Always prefer the date that appears first in the document, because headers are usually authoritative and footers are often boilerplate text copied from templates.
A fixed preference will be wrong whenever the footer is the corrected value.

B. **[CORRECT]** Include a conflict_detected flag, record both values with their locations, and route the record to review instead of silently choosing one.
Correct. Conflicting source data should be surfaced, not hidden.

C. **[INCORRECT]** Make the field required and add a validator that rejects any date not in ISO format.
Both dates are valid ISO dates, so the validator wouldn't catch the conflict.

D. **[INCORRECT]** Retry the extraction until the model returns a consistent date for the field.
Retrying may produce either date without addressing the conflict in the source.

Key concept: Handling conflicting source data (Task 4.4, 5.6).
Exam reasoning: Two valid but different dates mean the document is inconsistent, which needs a human decision.
Production lesson: Represent uncertainty in the data model.

Q58 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.3 | Intermediate | Single answer

Correct answer: A - Add an 'other' value with a document_type_detail string, and an 'unclear' value for ambiguous cases.

Why the correct answer is correct: Schema design should include enum values like 'unclear' for ambiguous cases and 'other' plus detail fields for extensible categories.

Option-by-option:

A. **[CORRECT]** Add an 'other' value with a document_type_detail string, and an 'unclear' value for ambiguous cases.
Correct. It keeps the enum extensible and avoids forced mislabeling.

B. **[INCORRECT]** Reject any document whose type is not in the enum and drop it from the batch.
Dropping documents silently loses data.

C. **[INCORRECT]** Change document_type to free text so the model can write whatever it wants.
Free text loses categorical consistency that downstream systems rely on.

D. **[INCORRECT]** Add a new enum value for every document type the team can imagine, and update it each quarter.
The list can never be complete, and unforeseen types would still be forced into existing values.

Key concept: Enum extensibility (Task 4.3).
Exam reasoning: Forced mislabeling arises from a closed set that cannot represent the document.
Production lesson: Give the model an honest way to say 'something else'.

Q59 | Domain 5: Context Management & Reliability | Scenario 6: Structured Data Extraction

Task statement 5.5 | Challenging | Single answer

Correct answer: C - Measure accuracy segmented by document type and field, including low-volume types such as handwritten receipts, before reducing human review.

Why the correct answer is correct: Aggregate accuracy metrics can mask poor performance on specific document types or fields. Accuracy should be validated by document type and field segment before automating high-confidence extractions.

Option-by-option:

A. **[INCORRECT]** Raise the confidence threshold slightly and proceed with full automation, since 97% accuracy is above the usual target for automated extraction.
A single aggregate number says nothing about the weakest segments.

B. **[INCORRECT]** Ask the model to rate its own confidence for the whole batch and automate if the average is high.
Self-reported averages hide segment-level weaknesses and are poorly calibrated.

C. **[CORRECT]** Measure accuracy segmented by document type and field, including low-volume types such as handwritten receipts, before reducing human review.
Correct. Aggregate accuracy can hide weak segments.

D. **[INCORRECT]** Remove review for typed invoices and keep it for everything else, based on the overall accuracy.
The decision should come from per-segment results, not from the aggregate.

Key concept: Segmented accuracy analysis (Task 5.5).
Exam reasoning: The validation set is dominated by one document type, so the overall number is not representative.
Production lesson: Look at the tail, not only the average.

Q60 | Domain 5: Context Management & Reliability | Scenario 6: Structured Data Extraction

Task statement 5.5 | Intermediate | Single answer

Correct answer: A - Stratified random sampling of high-confidence extractions for ongoing human review, to measure error rates and catch novel patterns.

Why the correct answer is correct: Stratified random sampling of high-confidence extractions supports ongoing error-rate measurement and detection of novel error patterns.

Option-by-option:

A. **[CORRECT]** Stratified random sampling of high-confidence extractions for ongoing human review, to measure error rates and catch novel patterns.
Correct. Ongoing sampling detects drift in the population you otherwise never look at.

B. **[INCORRECT]** Re-running the original validation set every quarter and confirming the accuracy is unchanged.
The old set doesn't contain the new vendor layout.

C. **[INCORRECT]** Lowering the confidence threshold every month by a small fixed amount to keep up with new formats.
Moving the threshold without data is arbitrary and doesn't detect errors.

D. **[INCORRECT]** Reviewing only low-confidence extractions, since they are where the model has already signaled uncertainty.
That never examines the auto-accepted records where the drift occurred.

Key concept: Ongoing quality monitoring (Task 5.5).
Exam reasoning: The failures hid within auto-accepted records, so only sampling those can reveal them.
Production lesson: Audit what you trust, continuously.

---

7. Domain-level performance review guide

Use your tracker. A domain below roughly 70% on a paper deserves a focused review before the next paper. Look at the task statements and concepts listed for each domain and re-read the matching sections of the official Exam Guide.

Domain 1 - Agentic Architecture & Orchestration (27% official weight; 16 questions here)

Task	Topic	Questions
1.1	Agentic loops for autonomous task execution	2
1.4	Multi-step workflows: enforcement and handoff	5
1.5	Agent SDK hooks: interception and normalization	1, 3, 7, 46, 47
1.6	Task decomposition strategies	4, 6, 17, 32, 33, 34, 48
1.7	Session state, resumption, and forking	16, 18

Lessons to carry forward: When a rule can be expressed as a check on tool arguments, enforce it in code and give the agent a safe alternative path. Treat tool calls as a list, not a single value. Pair each request with its result. Minimize what enters the context. Data the model never sees cannot be leaked by it. Use the least dynamic structure that fits the task. Every retried write needs an idempotency story. Return what you know, say what you don't, and offer a next step.

Domain 2 - Tool Design & MCP Integration (18% official weight; 11 questions here)

Task	Topic	Questions
2.1	Tool interfaces: descriptions and boundaries	50, 52
2.2	Structured error responses for MCP tools	8, 35, 36
2.3	Tool distribution and tool_choice	49, 51
2.4	MCP server integration (Claude Code and agents)	9, 21
2.5	Built-in tools: Read, Write, Edit, Bash, Grep, Glob	19, 20

Lessons to carry forward: Make failure impossible to confuse with absence. Build what differentiates you. Reuse what doesn't. Match the tool to what you are searching: names with Glob, content with Grep. Follow the trail, don't read the forest. When a shared configuration fails for one person, check what is not shared: their environment. Tell the agent whether trying again can help.

Domain 3 - Claude Code Configuration & Workflows (20% official weight; 12 questions here)

Task	Topic	Questions
3.1	CLAUDE.md hierarchy, scoping, modularity	22, 25
3.2	Custom slash commands and skills	27
3.3	Path-specific rules	24, 26
3.4	Plan mode vs direct execution	28
3.5	Iterative refinement techniques	23, 29
3.6	Claude Code in CI/CD pipelines	37, 38, 39, 40

Lessons to carry forward: Place instructions at the narrowest scope where they apply. Give the model the whole picture when the problems are one problem. Test glob patterns against your deepest paths, and check with /memory what has loaded. Observe before you change. Check what actually loaded. Load context only when it is relevant. Put usage guidance where it appears at invocation.

Domain 4 - Prompt Engineering & Structured Output (20% official weight; 12 questions here)

Task	Topic	Questions
4.1	Explicit criteria; reducing false positives	10
4.3	Structured output via tool use and JSON schemas	42, 44, 56, 58
4.4	Validation, retry, feedback loops	41, 53, 55, 57
4.5	Batch processing (Message Batches API)	43, 45, 54

Lessons to carry forward: Define categories so two reviewers, human or model, would agree. Tell the model exactly what failed. Validate meaning separately from shape. Know both when batch fits and what it cannot do. Design schemas that allow honest

answers. Compute SLAs from upper bounds.

Domain 5 - Context Management & Reliability (15% official weight; 9 questions here)

Task	Topic	Questions
5.1	Context management across long interactions	14, 15
5.2	Escalation and ambiguity resolution	11, 12, 13
5.4	Context in large codebase exploration	30, 31
5.5	Human review workflows; confidence calibration	59, 60

Lessons to carry forward: Agents should not invent policy. They should surface its gaps. Ask before acting when identity is ambiguous. Distinguish a request for a person from an expression of emotion. Structure long inputs as summary first, details labelled after. Design agent outputs for their consumer, not for the producer's convenience. Keep the coordinator's context for decisions, not for raw discovery.

8. Final revision checklist

Tick each item when you can explain it without looking at the key. Items are drawn from the concepts tested in this paper.

Done	Concept	First seen
[ ]	Tool call interception hooks versus prompt-based enforcement	Q1
[ ]	Agentic loop mechanics and tool results in history	Q2
[ ]	PostToolUse hooks for transformation and data minimization	Q3
[ ]	Choosing between prompt chaining and dynamic decomposition	Q4
[ ]	Deterministic safeguards against duplicate side effects	Q5
[ ]	Decomposition with partial failure handling	Q6
[ ]	Choosing hooks versus prompts	Q7
[ ]	Access failures versus valid empty results	Q8
[ ]	Choosing community versus custom MCP servers	Q9
[ ]	Explicit severity criteria	Q10
[ ]	Escalating on policy gaps	Q11
[ ]	Ambiguity resolution with multiple matches	Q12
[ ]	Escalation versus resolution under frustration	Q13
[ ]	Position-aware input ordering	Q14
[ ]	Structured upstream outputs for limited context	Q15
[ ]	Named session resumption	Q16
[ ]	Adaptive decomposition for open-ended tasks	Q17
[ ]	Resumption, fresh start, and forking	Q18
[ ]	Selecting Grep versus Glob	Q19
[ ]	Incremental code exploration	Q20
[ ]	MCP configuration and connection troubleshooting	Q21
[ ]	CLAUDE.md hierarchy and directory scope	Q22
[ ]	Single versus sequential issue feedback	Q23
[ ]	Glob patterns in path-scoped rules	Q24
[ ]	Using /memory to diagnose configuration	Q25
[ ]	Benefit of conditional rules	Q26
[ ]	Skill frontmatter: argument-hint	Q27
[ ]	Plan mode versus direct execution	Q28
[ ]	Test-driven iteration	Q29
[ ]	Subagent delegation for exploration	Q30
[ ]	/compact and scratchpad files	Q31
[ ]	Matching decomposition to workflow	Q32
[ ]	Multi-pass review design	Q33
[ ]	Selecting decomposition patterns	Q34
[ ]	Permission errors and retryability	Q35
[ ]	Failure versus empty success	Q36
[ ]	CLAUDE.md for CI context	Q37
[ ]	Incremental CI review and test generation	Q38
[ ]	Structured CI output	Q39
[ ]	Session context isolation	Q40

Done	Concept	First seen
[ ]	Validation-retry loops	Q41
[ ]	Schema validity versus semantic correctness	Q42
[ ]	Batch API suitability and limits	Q43
[ ]	Schema design for structured output	Q44
[ ]	Batch submission frequency and SLA calculation	Q45
[ ]	Hooks for normalization and gating	Q46
[ ]	Threshold enforcement with redirect	Q47
[ ]	Prompt chaining for long documents	Q48
[ ]	tool_choice behavior	Q49
[ ]	Eliminating tool overlap	Q50
[ ]	Forced tool selection for ordering	Q51
[ ]	Tool description boundaries and edge cases	Q52
[ ]	Limits of retry and nullable fields	Q53
[ ]	Batch failure handling	Q54
[ ]	Semantic validation and discrepancy flags	Q55
[ ]	What schemas do and do not guarantee	Q56
[ ]	Handling conflicting source data	Q57
[ ]	Enum extensibility	Q58
[ ]	Segmented accuracy analysis	Q59
[ ]	Ongoing quality monitoring	Q60

Before the next paper: classify each wrong answer as (a) knowledge gap, (b) misread of the stem or qualifier, or (c) a plausible-distractor trap. See CCAF_Final_Revision_Guide.pdf for the method.